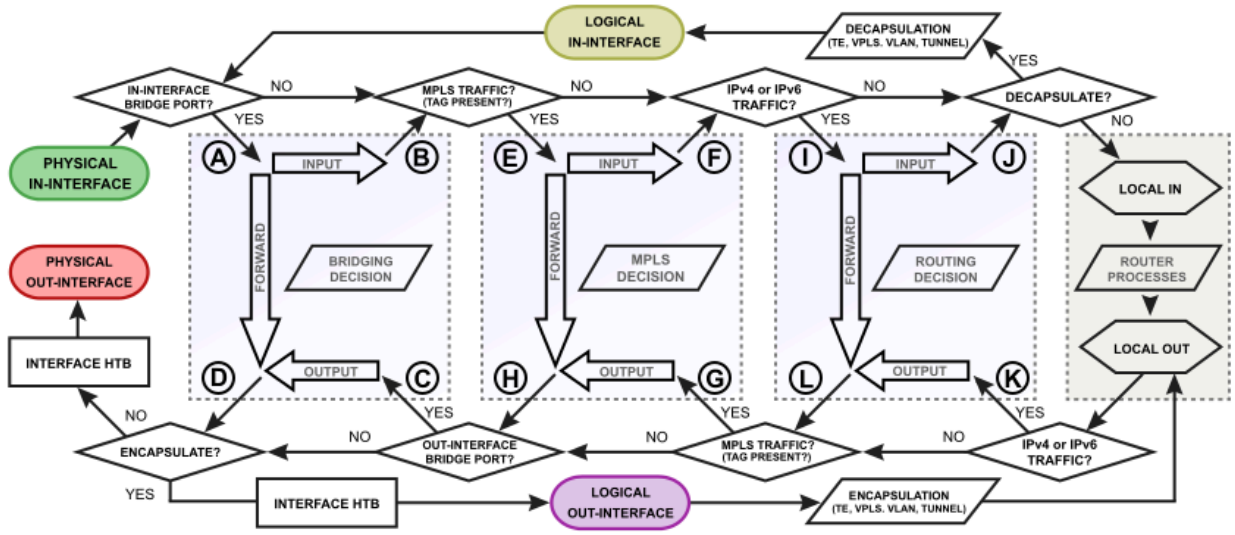
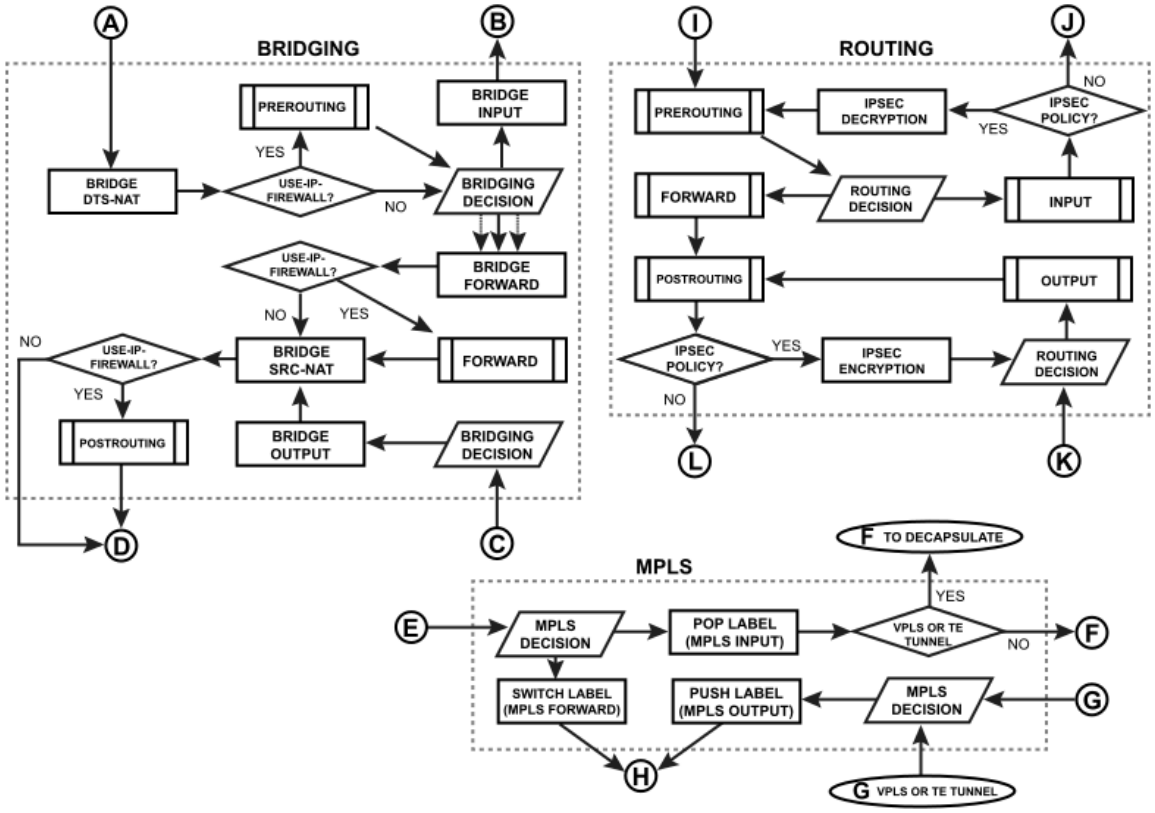
MikroTik Certified Traffic Control Engineer (MTCTCE)

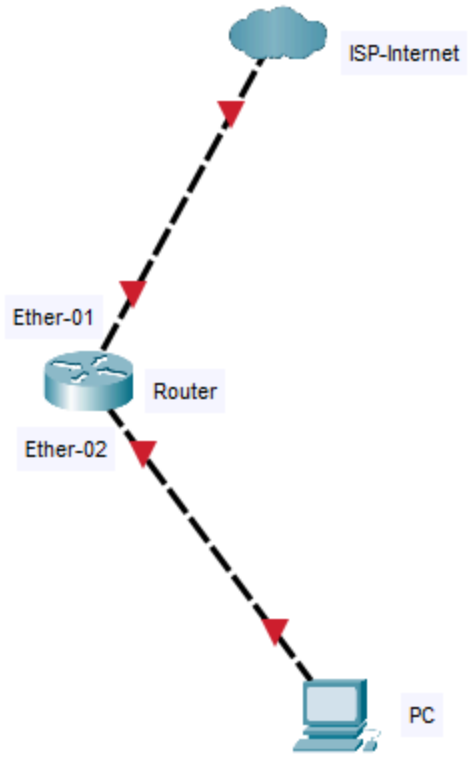
Packet flow diagram



Bridging and routing

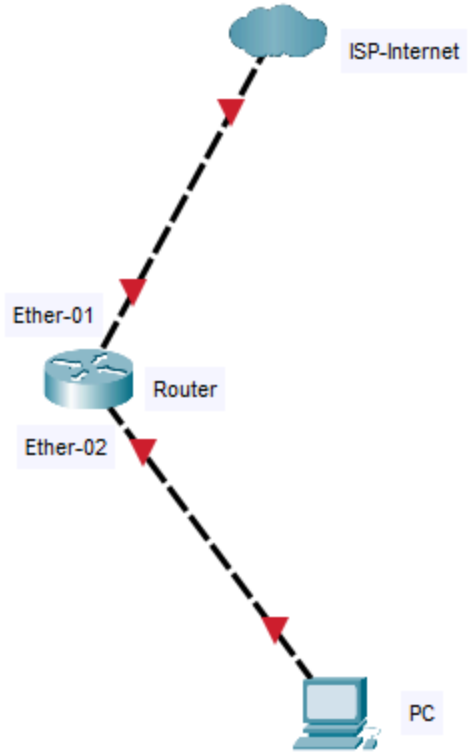

A
BRIDGING
B
PREROUTING
BRIDGE INPUT
YES
BRIDGE DTS-NAT
USE-IP-FIREWALL?
NO
BRIDGING DECISION
USE-IP-FIREWALL?
BRIDGE FORWARD
NO
YES
NO
USE-IP-FIREWALL?
BRIDGE SRC-NAT
FORWARD
YES
POSTROUTING
BRIDGE OUTPUT
BRIDGING DECISION
D
C
I
ROUTING
J
PREROUTING
IPSEC DECRYPTION
NO
IPSEC POLICY?
YES
FORWARD
ROUTING DECISION
INPUT
POSTROUTING
OUTPUT
IPSEC POLICY?
YES
IPSEC ENCRYPTION
ROUTING DECISION
NO
L
K
F TO DECAPSULATE
MPLS
YES
E
MPLS DECISION
POP LABEL (MPLS INPUT)
VPLS OR TE TUNNEL
NO
F
SWITCH LABEL (MPLS FORWARD)
PUSH LABEL (MPLS OUTPUT)
MPLS DECISION
G
H
G VPLS OR TE TUNNEL

IP link Configuration

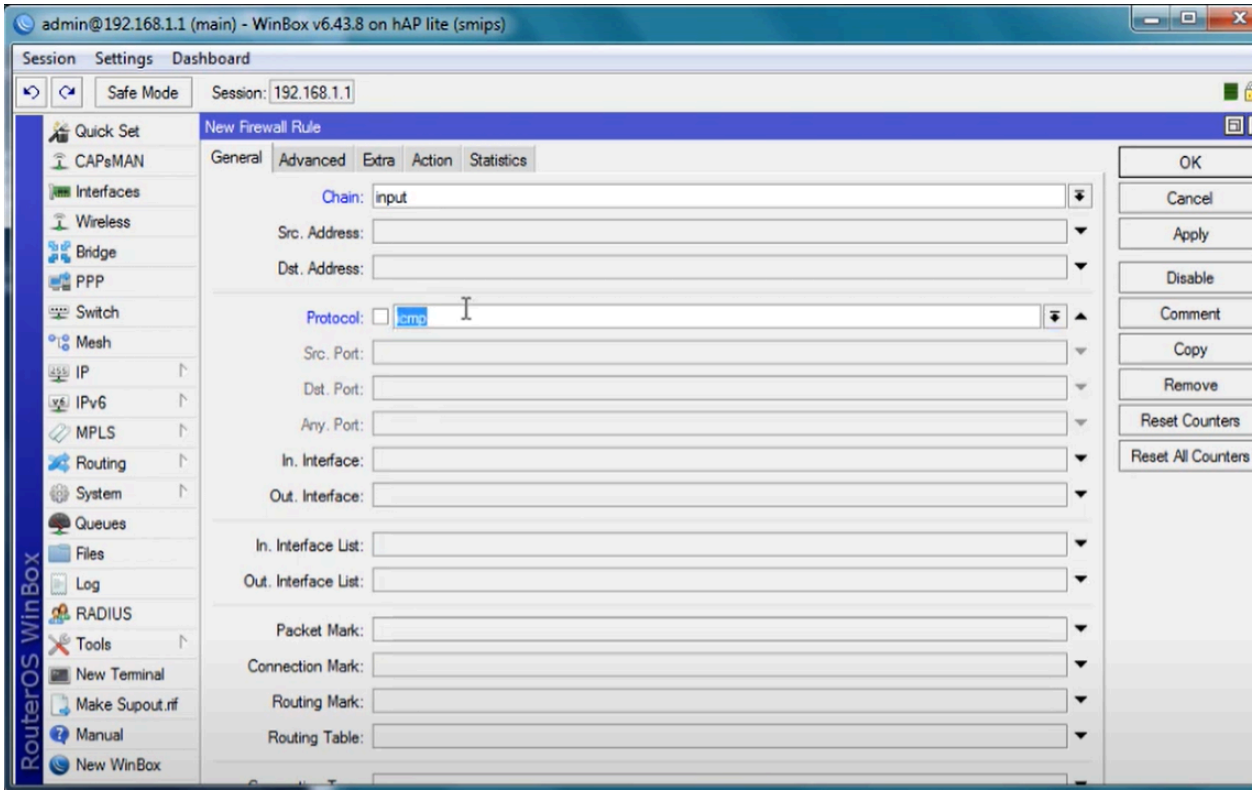


Firewall filter/nat/mangle

Filter + LAB
Input chain
Forward Chain
Output Chain



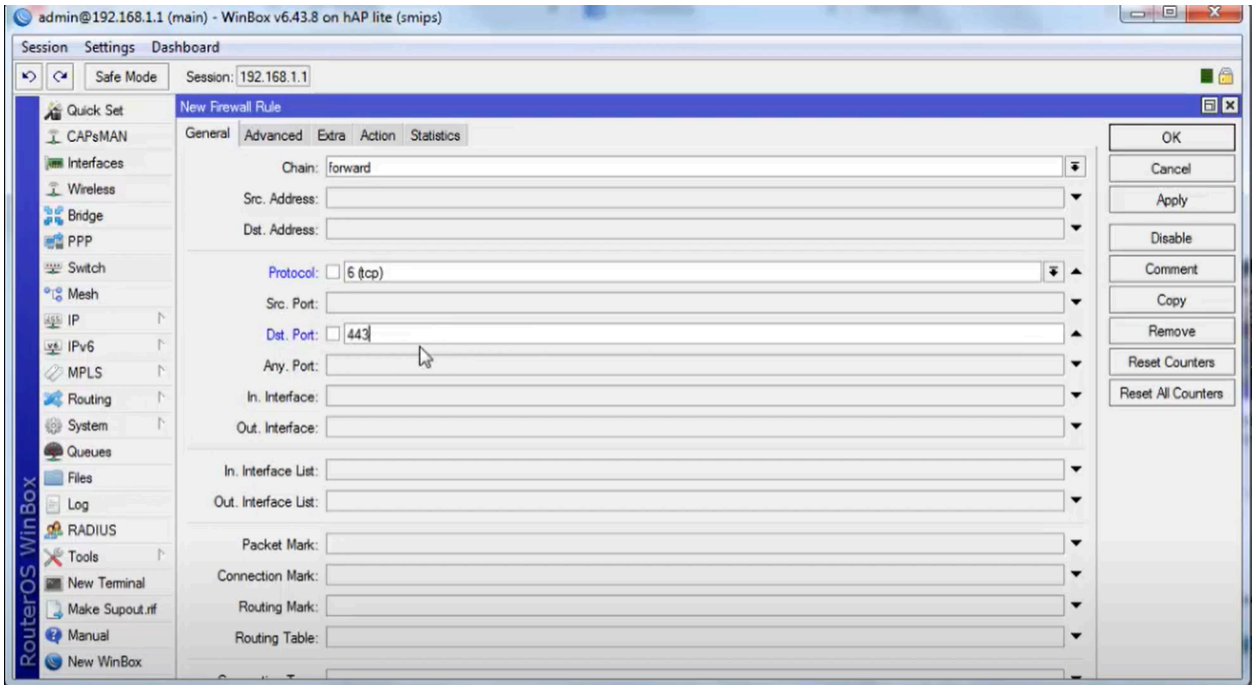
Input chain
/ip firewall filter
add action=drop chain=input protocol=icmp src-address=192.168.200.254



Forward

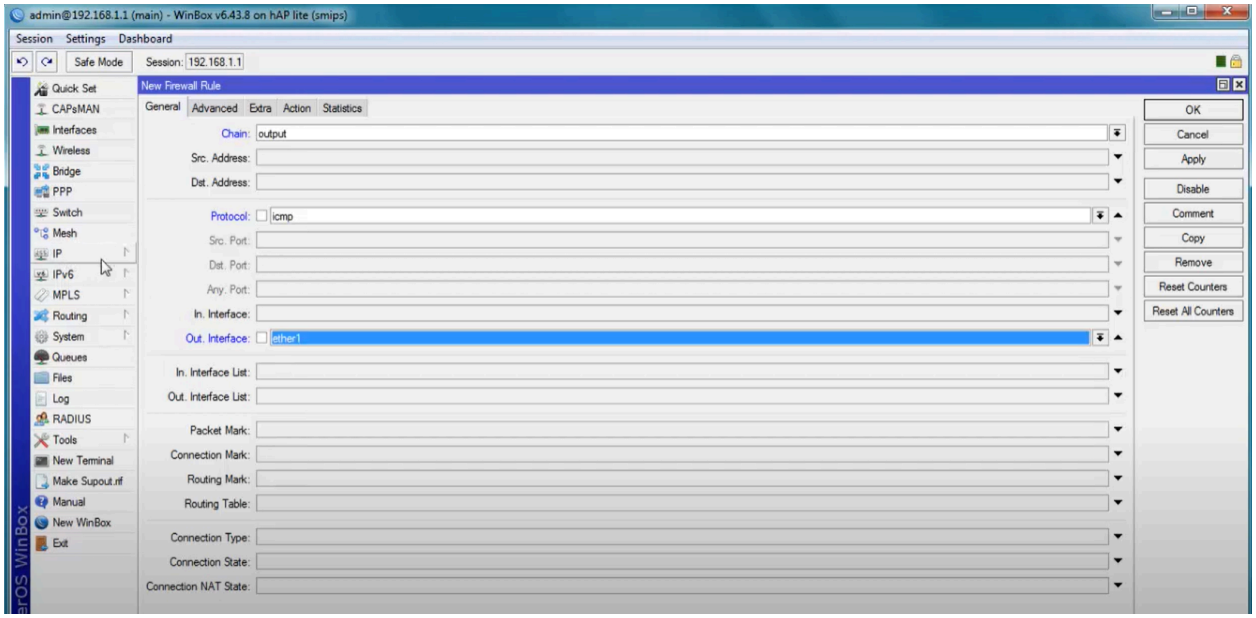
/ip firewall filter

add action=drop chain=forward dst-port=443 protocol=tcp



Output

/ip firewall filter
add action=drop chain=output out-interface=ether1 protocol=icmp



Action to take if packet is matched by the rule:

- `accept` - accept the packet. Packet is not passed to next firewall rule.
- `add-dst-to-address-list` - add destination address to address list specified by `address-list` parameter
- `add-src-to-address-list` - add source address to address list specified by `address-list` parameter
- `drop` - silently drop the packet
- `fasttrack-connection` - process packets from a connection using FastPath by enabling FastTrack for the connection
- `jump` - jump to the user defined chain specified by the value of `jump-target` parameter
- `log` - add a message to the system log containing following data: in-interface, out-interface, src-mac, protocol, src-ip:port->dst-ip:port and length of the packet. After packet is matched it is passed to next rule in the list, similar as `passthrough`
- `passthrough` - if packet is matched by the rule, increase counter and go to next rule (useful for statistics)
- `reject` - drop the packet and send an ICMP reject message
- `return` - passes control back to the chain from where the jump took place
- `tarpit` - captures and holds TCP connections (replies with SYN/ACK to the inbound TCP SYN packet)

Tarpit

Example

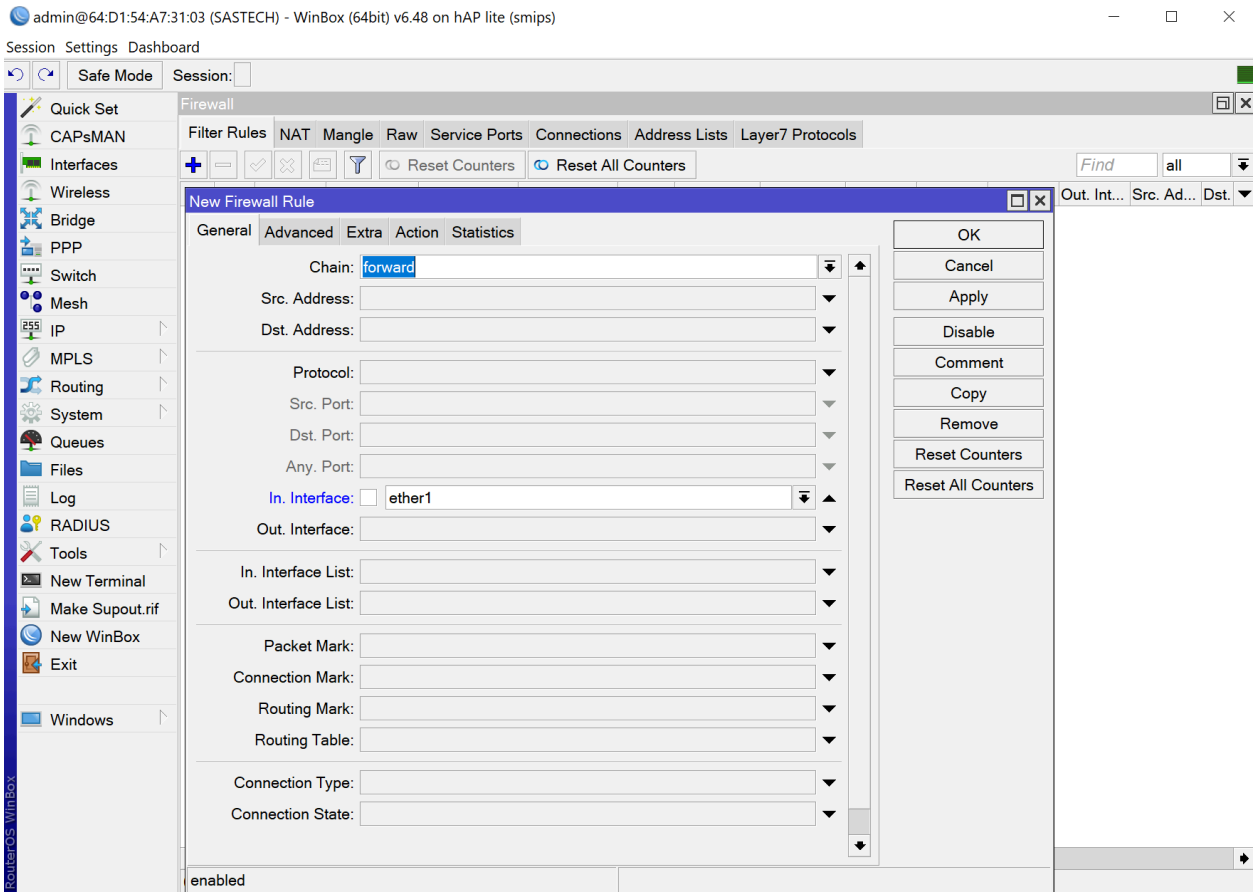
/ip firewall filter add chain=input protocol=tcp src-address-list=blocked-addr \

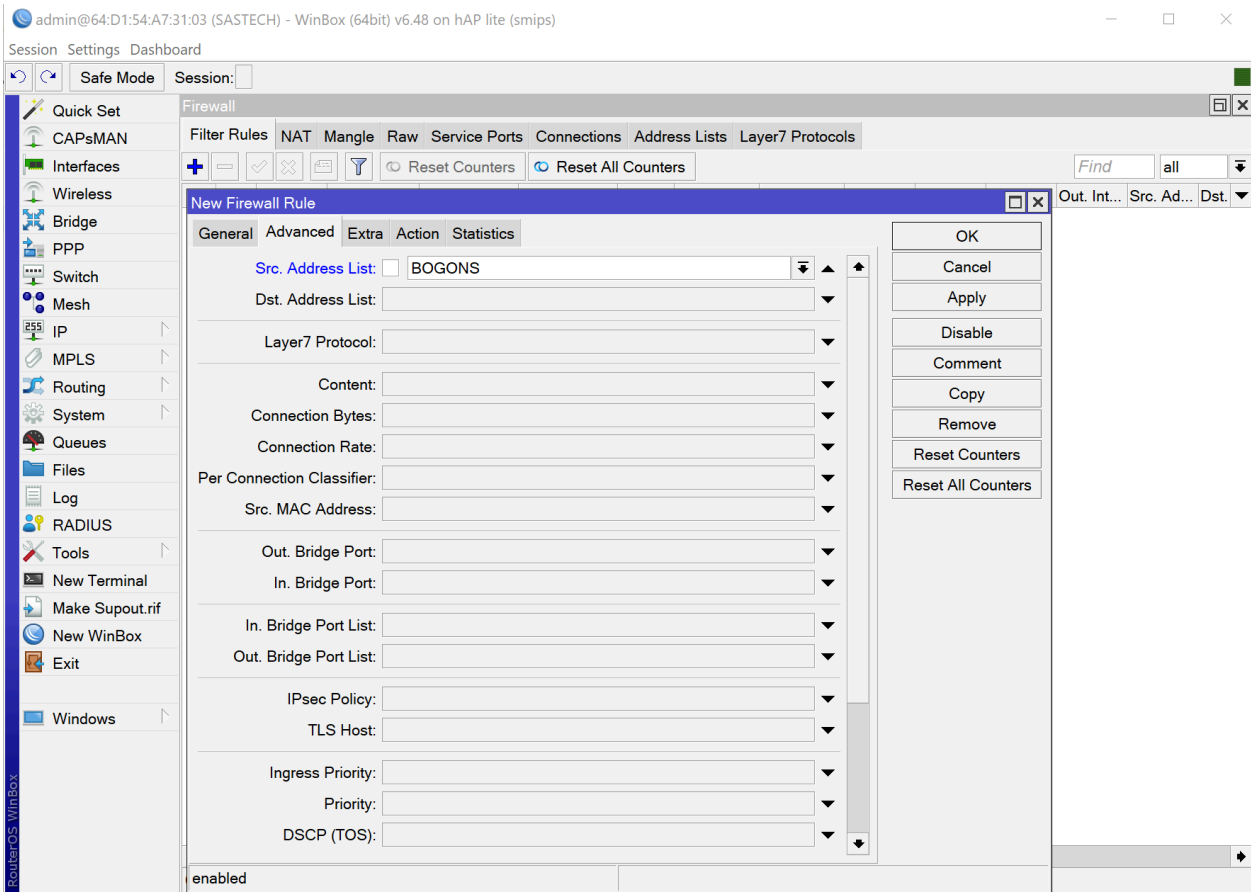
connection-limit=3,32 action=tarpit

FastTrack connection:
FastTracked packets bypass firewall, connection tracking, simple queues
Example:

/ip firewall filter add chain=forward action=fasttrack-connection
connection-state=established,related
/ip firewall filter add chain=forward action=accept connection-state=established,related

Address List
/ip firewall address-list
add list="BOGONS" address=0.0.0.0/8
add list="BOGONS" address=10.0.0.0/8
add list="BOGONS" address=100.64.0.0/10
add list="BOGONS" address=127.0.0.0/8
add list="BOGONS" address=169.254.0.0/16
add list="BOGONS" address=172.16.0.0/12
add list="BOGONS" address=192.0.0.0/24
add list="BOGONS" address=192.0.2.0/24
add list="BOGONS" address=192.168.0.0/16
add list="BOGONS" address=198.18.0.0/15
add list="BOGONS" address=198.51.100.0/24
add list="BOGONS" address=203.0.113.0/24
add list="BOGONS" address=224.0.0.0/3


admin@64:D1:54:A7:31:03 (SASTECH) - WinBox (64bit) v6.48 on hAP lite (smips)
Session Settings Dashboard
Safe Mode
Session:
Quick Set
CAPsMAN
Interfaces
Wireless
Bridge
PPP
Switch
Mesh
IP
MPLS
Routing
System
Queues
Files
Log
RADIUS
Tools
New Terminal
Make Supout.rif
New WinBox
Exit
Windows
RouterOS WinBox
Firewall
Filter Rules
NAT
Mangle
Raw
Service Ports
Connections
Address Lists
Layer7 Protocols
Reset Counters
Reset All Counters
Find
all
Out. Int...
Src. Ad...
Dst.
New Firewall Rule
General
Advanced
Extra
Action
Statistics
Chain:
forward
Src. Address:
Dst. Address:
Protocol:
Src. Port:
Dst. Port:
Any. Port:
In. Interface:
ether1
Out. Interface:
In. Interface List:
Out. Interface List:
Packet Mark:
Connection Mark:
Routing Mark:
Routing Table:
Connection Type:
Connection State:
OK
Cancel
Apply
Disable
Comment
Copy
Remove
Reset Counters
Reset All Counters
enabled



Custom chain
The jump action allows you to create a custom chain of firewall rules, so that you can have specific kinds of traffic go through extra processing without making all other kinds of traffic go through the same thing, thus reducing CPU time and increasing throughput on the router.

/ip firewall filter
add action=jump chain=forward jump-target=tcp protocol=tcp
add action=drop chain=tcp comment="deny TFTP" dst-port=69 protocol=tcp

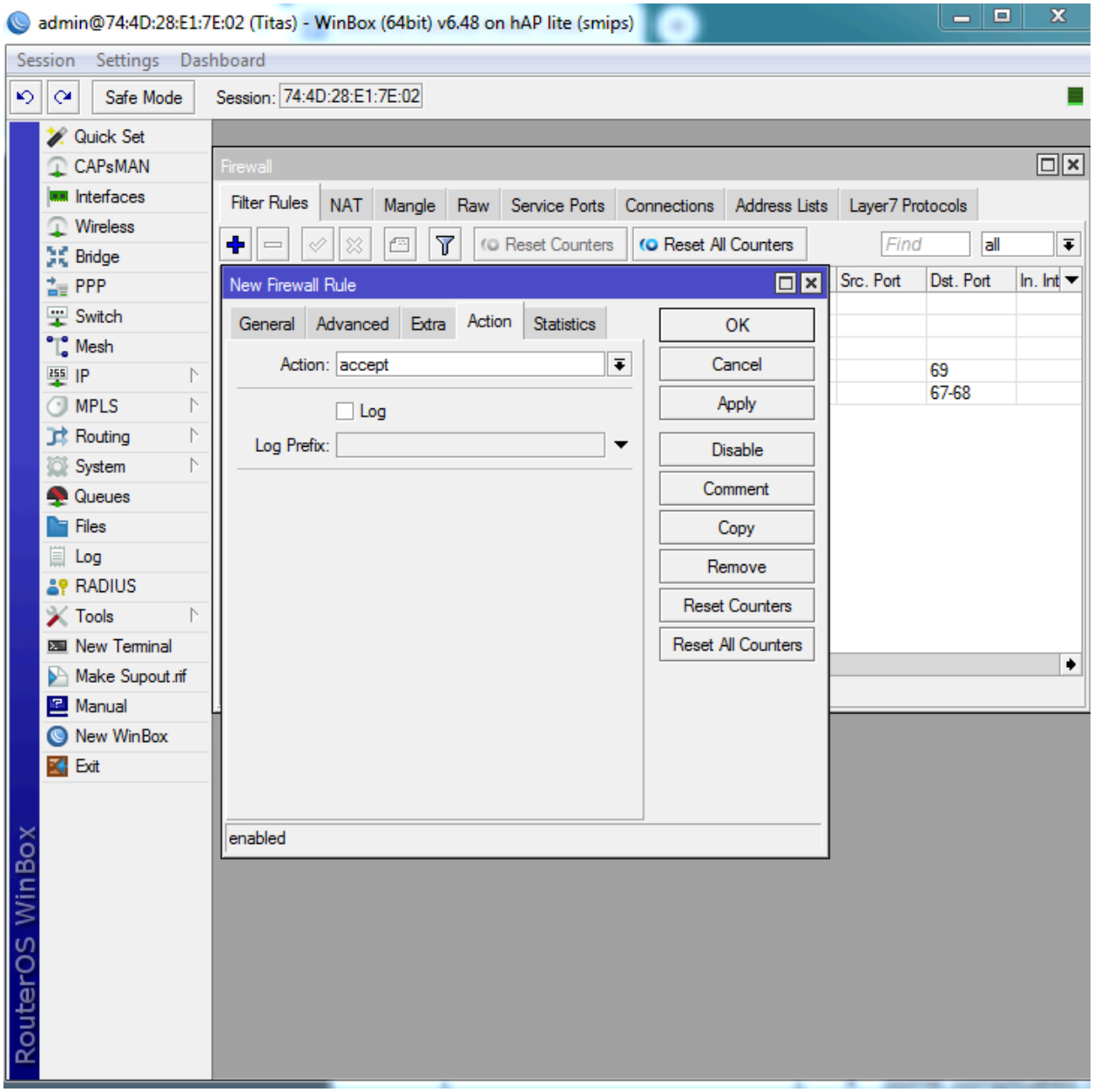
Log
/ip firewall filter
add action=log chain=input

add-src-to-address-list or action=add-dst-to-address-list
Firewall address lists allow a user to create lists of IP addresses grouped together under a common name. Firewall filter, mangle and NAT facilities can then use those address lists to match packets against them.

The address list records can also be updated dynamically via the action=add-src-to-address-list or action=add-dst-to-address-list items found in NAT, Mangle and Filter facilities.

Example:
```
/ip firewall address-list add list=drop_traffic address=192.0.34.166/32
/ip firewall mangle add action=add-src-to-address-list address-list=drop_traffic \
    address-list-timeout=5m chain=prerouting dst-port=23 protocol=tcp
/ip firewall filter add action=drop chain=input src-address-list=drop_traffic
```

all rule "actions" covered
Action to take if packet is matched by the rule:
accept - accept the packet. Packet is not passed to next firewall rule.
add-dst-to-address-list - add destination address to address list specified by address-list parameter
add-src-to-address-list - add source address to address list specified by address-list parameter
drop - silently drop the packet
fasttrack-connection - process packets from a connection using FastPath by enabling FastTrack for the connection
jump - jump to the user defined chain specified by the value of jump-target parameter
log - add a message to the system log containing following data: in-interface, out-interface, src-mac, protocol, src-ip:port->dst-ip:port and length of the packet. After packet is matched it is passed to next rule in the list, similar as passthrough
passthrough - if packet is matched by the rule, increase counter and go to next rule (useful for statistics)
reject - drop the packet and send an ICMP reject message
return - passes control back to the chain from where the jump took place
tarpit - captures and holds TCP connections (replies with SYN/ACK to the inbound TCP SYN packet)



most common rule "conditions" covered
Accept, drop

NAT chains

Source NAT

Masquerade

If you want to "hide" the private LAN 192.168.0.0/24 "behind" one address 10.5.8.109 given to you by the ISP, you should use the source network address translation (masquerading) feature of the MikroTik router. The masquerading will change the source IP address and port of the packets originated from the network 192.168.0.0/24 to the address 10.5.8.109 of the router when the packet is routed through it.

To use masquerading, a source NAT rule with action 'masquerade' should be added to the firewall configuration:

```
/ip firewall nat add chain=srcnat action=masquerade out-interface=Public
```

Destination NAT

Forward all traffic to internal host

If you want to link Public IP 10.5.8.200 address to Local one 192.168.0.109, you should use destination address translation feature of the MikroTik router. Also if you want allow Local server to initiate connections to outside with given Public IP you should use source address translation, too.

Add Public IP to Public interface:

```
/ip address add address=10.5.8.200/32 interface=Public
```

Add rule allowing access to the internal server from external networks:

```
/ip firewall nat add chain=dstnat dst-address=10.5.8.200 action=dst-nat \
        to-addresses=192.168.0.109
```

Add rule allowing the internal server to initate connections to the outer networks having its source address translated to 10.5.8.200:

```
/ip firewall nat add chain=srcnat src-address=192.168.0.109 action=src-nat \
        to-addresses=10.5.8.200
```

Port mapping/forwarding

If you would like to direct requests for a certain port to an internal machine (sometimes called opening a port, port mapping), you can do it like this:

```
/ip firewall nat add chain=dstnat dst-port=1234 action=dst-nat protocol=tcp
to-address=192.168.1.1 to-port=1234
```

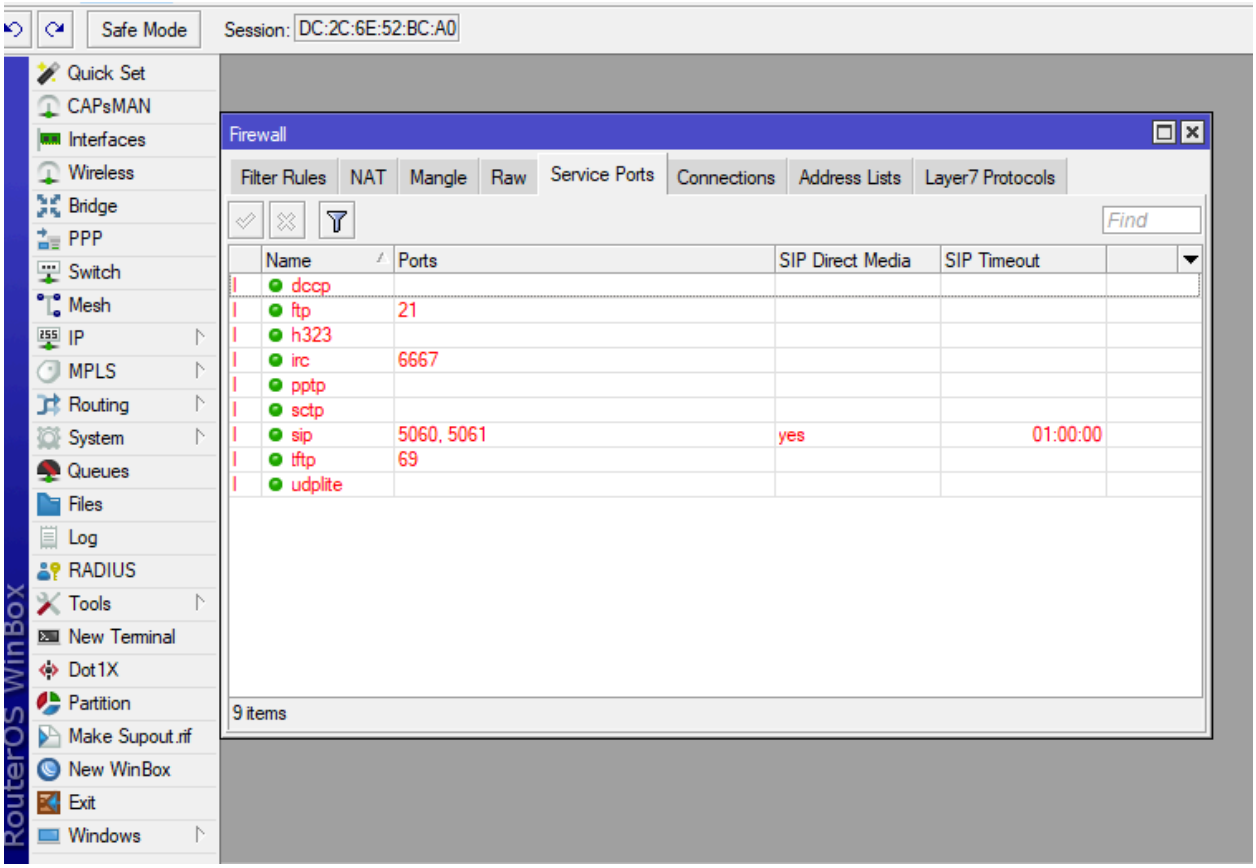
all rule "actions" covered

Action to take if packet is matched by the rule:

- `accept` - accept the packet. Packet is not passed to next NAT rule.
- `add-dst-to-address-list` - add destination address to Address list specified by `address-list` parameter
- `add-src-to-address-list` - add source address to Address list specified by `address-list` parameter
- `dst-nat` - replaces destination address and/or port of an IP packet to values specified by `to-addresses` and `to-ports` parameters
- `jump` - jump to the user defined chain specified by the value of `jump-target` parameter
- `log` - add a message to the system log containing following data: in-interface, out-interface, src-mac, protocol, src-ip:port->dst-ip:port and length of the packet. After packet is matched it is passed to next rule in the list, similar as `passthrough`
- `masquerade` - replaces source port of an IP packet to one specified by `to-ports` parameter and replace source address of an IP packet to IP determined by routing facility. `Read more >>`
- `netmap` - creates a static 1:1 mapping of one set of IP addresses to another one. Often used to distribute public IP addresses to hosts on private networks
- `passthrough` - if packet is matched by the rule, increase counter and go to next rule (useful for statistics).
- `redirect` - replaces destination port of an IP packet to one specified by `to-ports` parameter and destination address to one of the router's local addresses
- `return` - passes control back to the chain from where the jump took place
- `same` - gives a particular client the same source/destination IP address from supplied range for each connection. This is most frequently used for services that expect the same client address for multiple connections from the same client
- `src-nat` - replaces source address of an IP packet to values specified by `to-addresses` and `to-ports` parameters

NAT helpers
Hosts behind a NAT-enabled router do not have true end-to-end connectivity. Therefore some Internet protocols might not work in scenarios with NAT. Services that require the initiation of TCP connection from outside the private network or stateless protocols such as UDP, can be disrupted. Moreover, some protocols are inherently incompatible with NAT, a bold example is **AH protocol from the IPsec suite**.

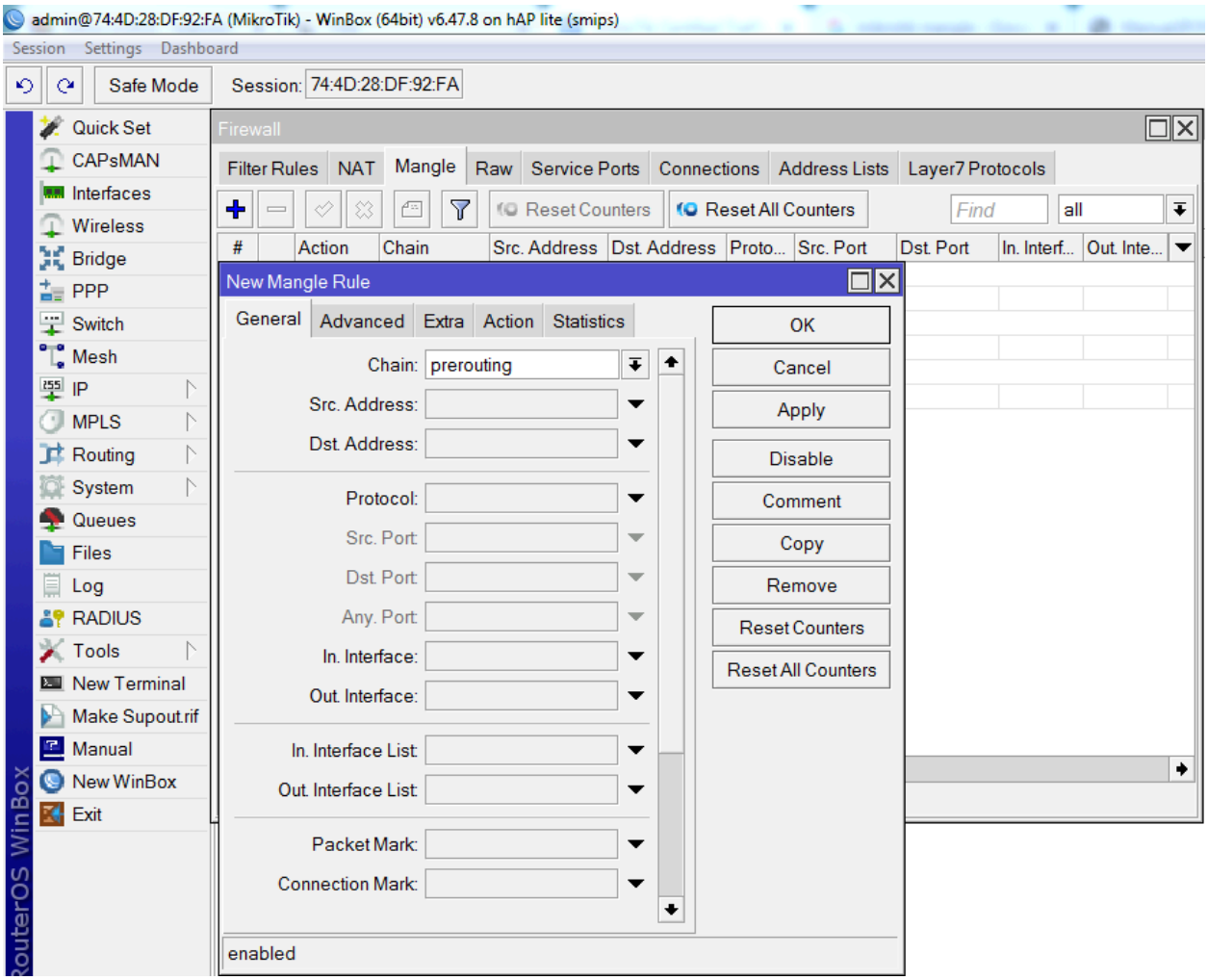
To overcome these limitations RouterOS includes a number of so-called NAT helpers, that enable NAT traversal for various protocols.
If connection tracking is not enabled then firewall service ports will be shown as inactive



Mangle

Mangle is a kind of 'marker' that marks packets for future processing with special marks. Many other facilities in RouterOS make use of these marks, e.g. queue trees, NAT, routing. They identify a packet based on its mark and process it accordingly. The mangle marks exist only within the router, they are not transmitted across the network.

Firewall mangle rules consist of five predefined chains that cannot be deleted:



Chains

The PREROUTING chain: Rules in this chain apply to packets as they just arrive on the network interface;

The INPUT chain: Rules in this chain apply to packets just before they're given to a local process;

The OUTPUT chain: The rules here apply to packets just after they've been produced by a process;

The FORWARD chain: The rules here apply to any packets that are routed through the current host;

The POSTROUTING chain: The rules in this chain apply to packets as they just leave the network interface;

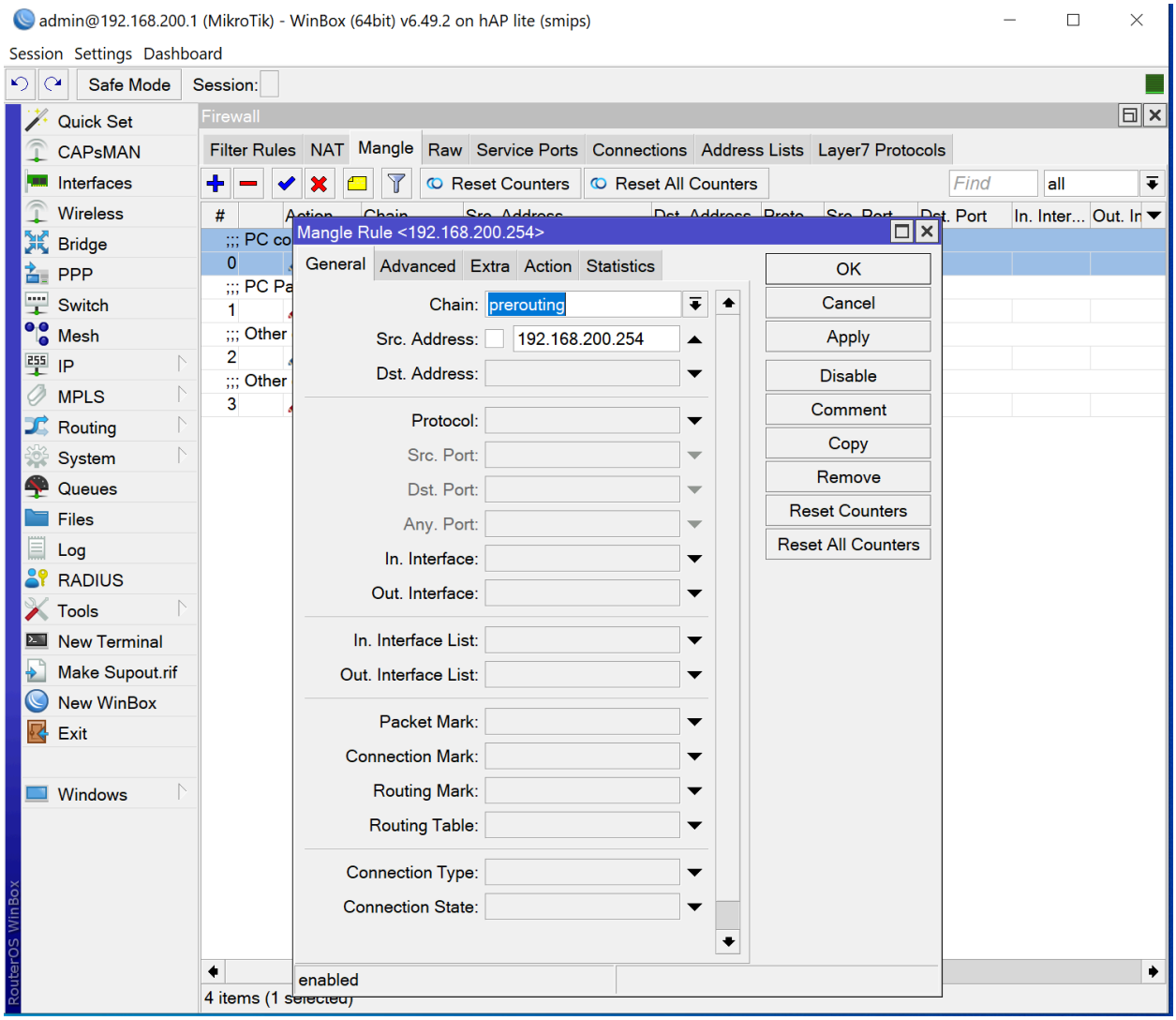
all rule "actions" covered

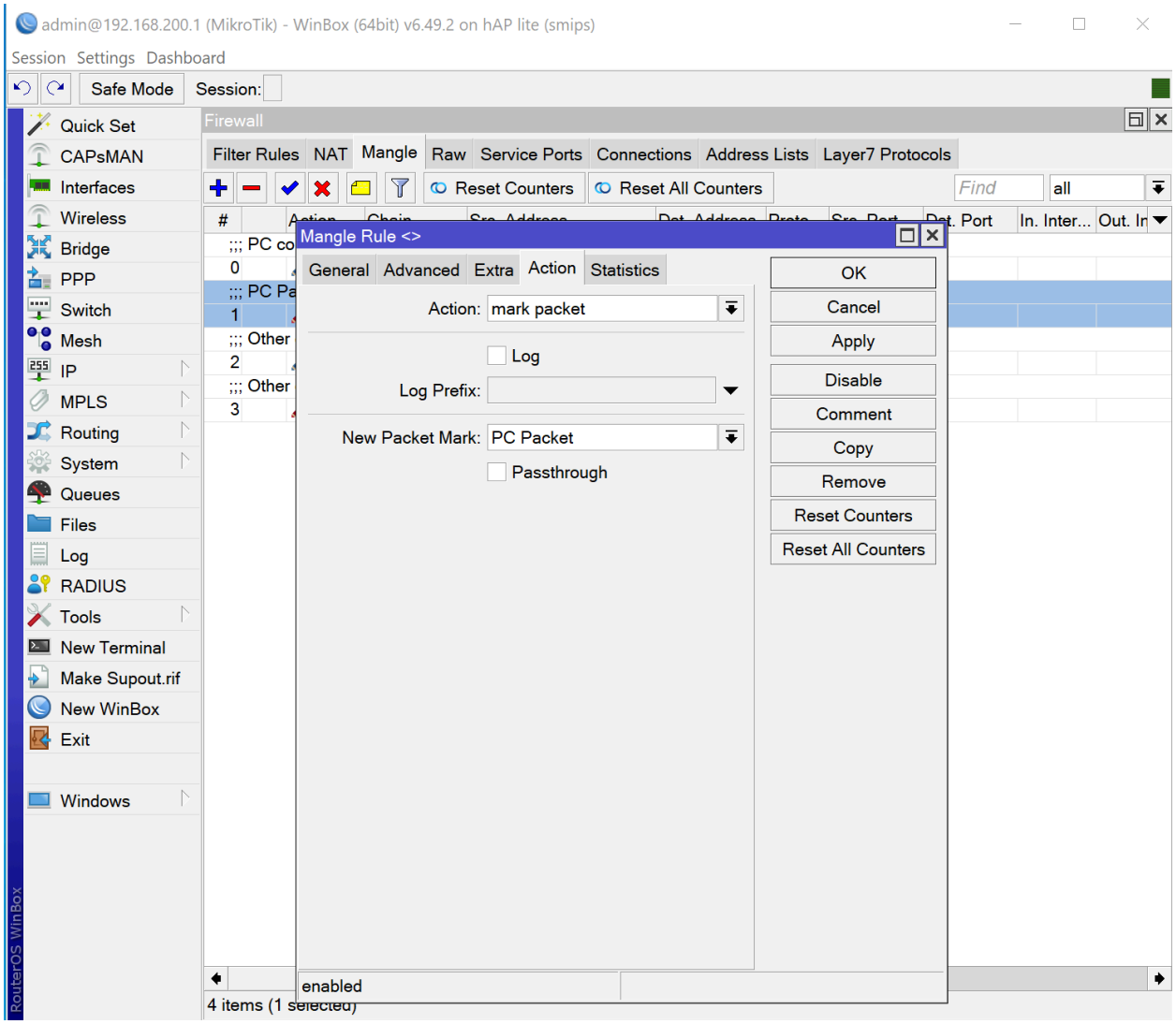
Action to take if a packet is matched by the rule:

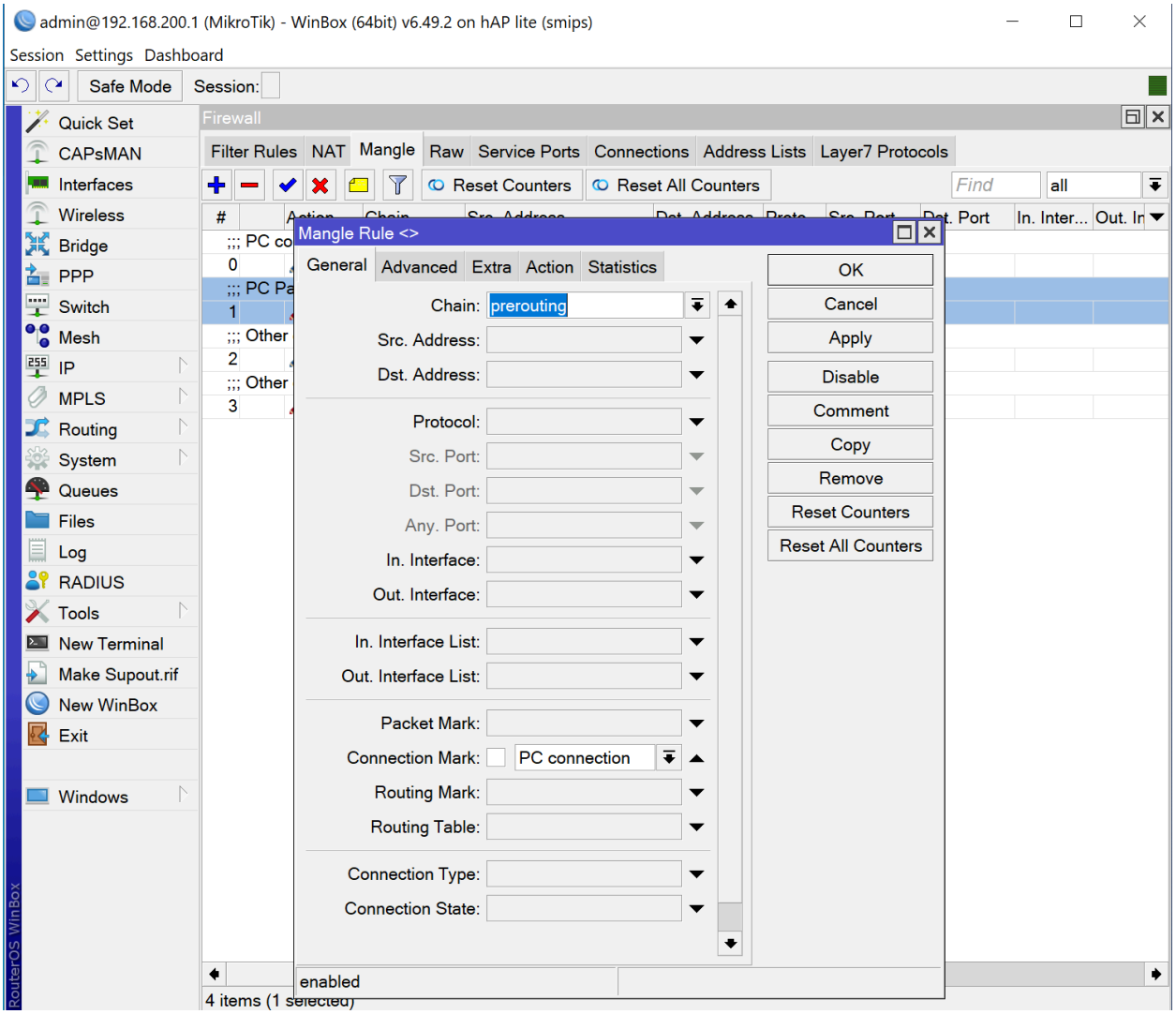
accept - accept the packet. A packet is not passed to the next firewall rule.

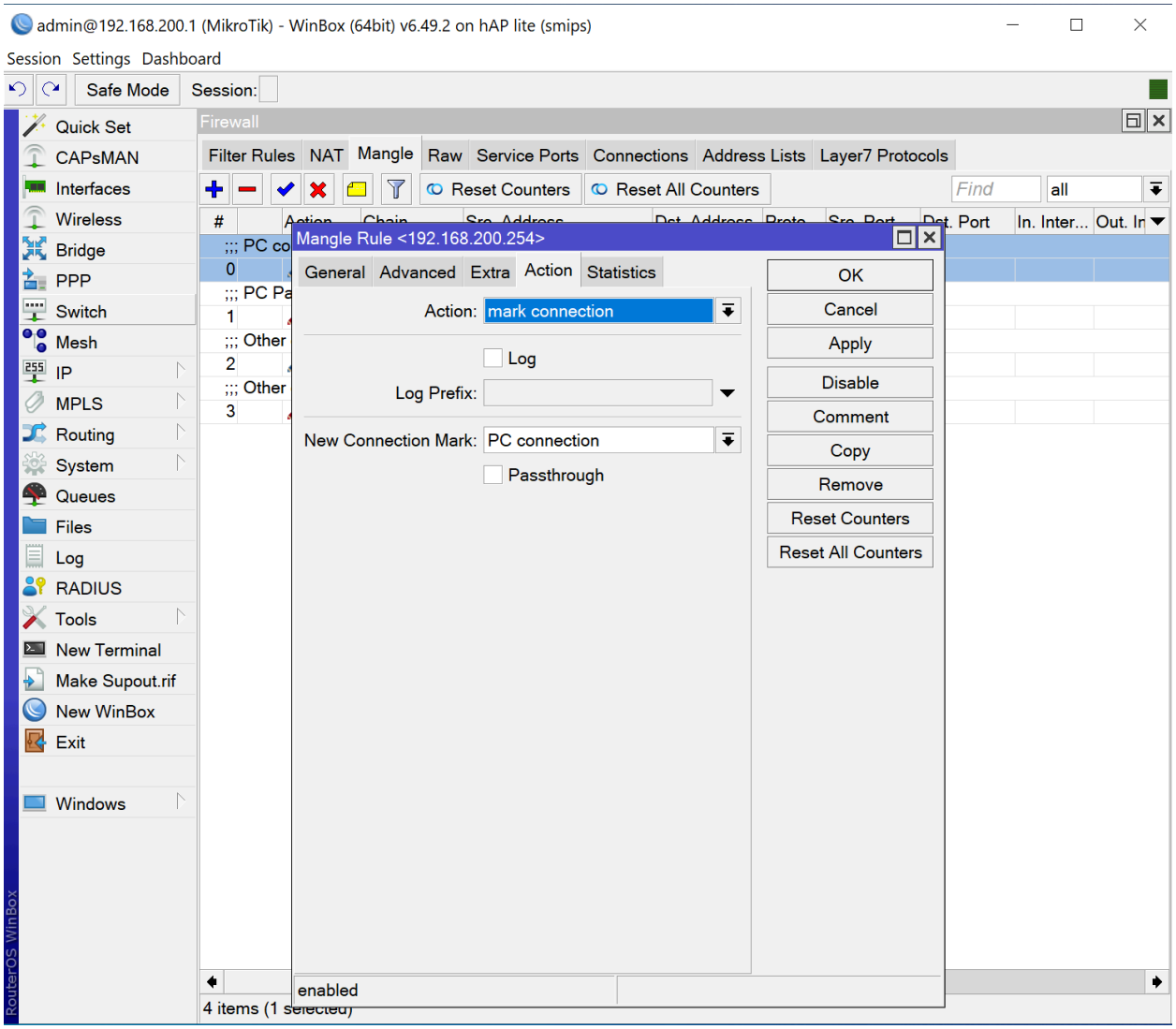
add-dst-to-address-list - add destination address to address list specified by address-list parameter
add-src-to-address-list - add source address to address list specified by address-list parameter
change-dscp - change Differentiated Services Code Point (DSCP) field value specified by the new-dscp parameter
change-mss - change Maximum Segment Size field value of the packet to a value specified by the new-mss parameter
change-ttl - change Time to Live field value of the packet to a value specified by the new-ttl parameter
clear-df - clear 'Do Not Fragment' Flag
fasttrack-connection - shows fasttrack counters, useful for statistics
jump - jump to the user-defined chain specified by the value of jump-target parameter
log - add a message to the system log containing the following data: in-interface, out-interface, src-mac, protocol, src-ip:port->dst-ip:port and length of the packet. After a packet is matched it is passed to the next rule in the list, similar as passthrough
mark-connection - place a mark specified by the new-connection-mark parameter on the entire connection that matches the rule
mark-packet - place a mark specified by the new-packet-mark parameter on a packet that matches the rule
mark-routing - place a mark specified by the new-routing-mark parameter on a packet. This kind of marks is used for policy routing purposes only
passthrough - if a packet is matched by the rule, increase counter and go to next rule (useful for statistics).
return - pass control back to the chain from where the jump took place
route - forces packets to a specific gateway IP by ignoring normal routing decision (prerouting chain only)
set-priority - set priority specified by the new-priority parameter on the packets sent out through a link that is capable of transporting priority (VLAN or WMM-enabled wireless interface). Read more>
sniff-pc - send a packet to a remote RouterOS CALEA server.
sniff-tzsp - send a packet to a remote TZSP compatible system (such as Wireshark). Set remote target with sniff-target and sniff-target-port parameters (Wireshark recommends port 37008)
strip-ipv4-options - strip IPv4 option fields from IP header, the action does not actually remove IPv4 options but rather replaces all option octets with NOP, further matcher with ipv4-options=any will still match the packet.

Connection mark and packet mark

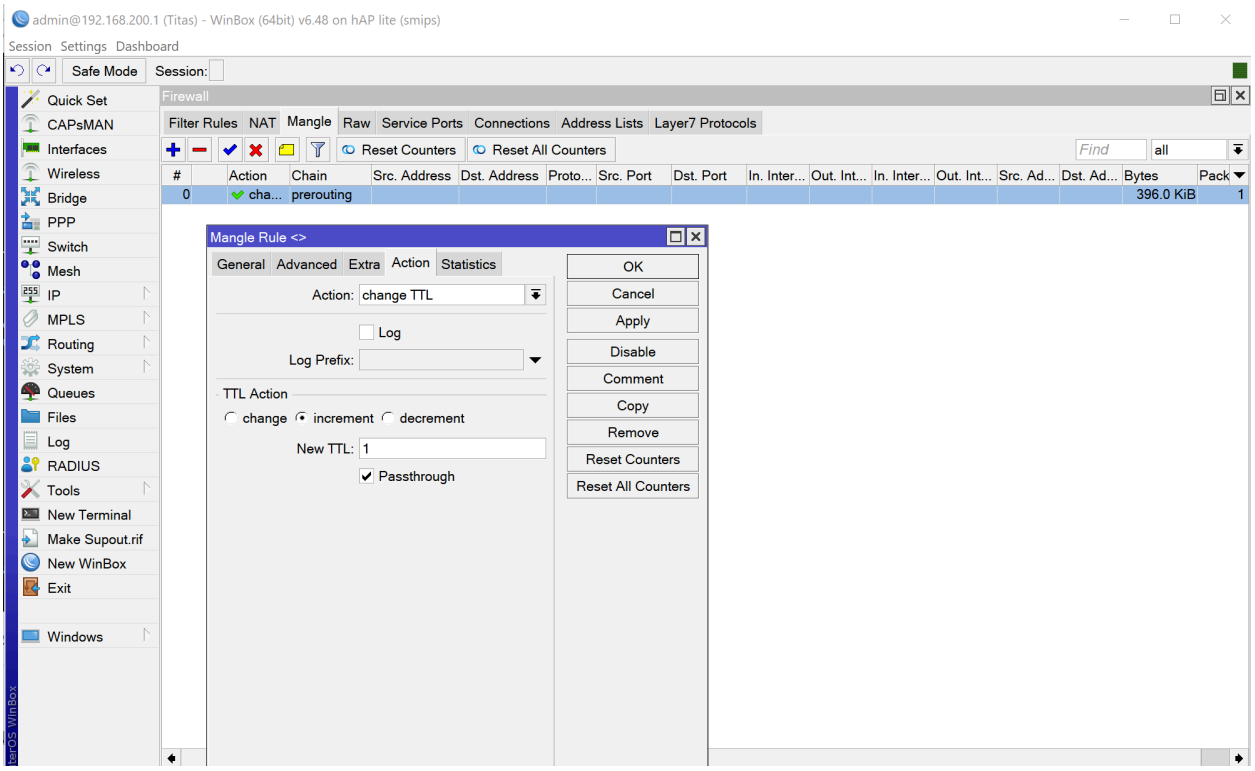

admin@192.168.200.1 (MikroTik) - WinBox (64bit) v6.49.2 on hAP lite (smips)
Session Settings Dashboard
Safe Mode
Session:
Quick Set
CAPsMAN
Interfaces
Wireless
Bridge
PPP
Switch
Mesh
IP
MPLS
Routing
System
Queues
Files
Log
RADIUS
Tools
New Terminal
Make Supout.rif
New WinBox
Exit
Windows
RouterOS WinBox
Firewall
Filter Rules
NAT
Mangle
Raw
Service Ports
Connections
Address Lists
Layer7 Protocols
Reset Counters
Reset All Counters
Find
all
Mangle Rule <192.168.200.254>
General
Advanced
Extra
Action
Statistics
Chain: prerouting
Src. Address: 192.168.200.254
Dst. Address:
Protocol:
Src. Port:
Dst. Port:
Any. Port:
In. Interface:
Out. Interface:
In. Interface List:
Out. Interface List:
Packet Mark:
Connection Mark:
Routing Mark:
Routing Table:
Connection Type:
Connection State:
OK
Cancel
Apply
Disable
Comment
Copy
Remove
Reset Counters
Reset All Counters
enabled
4 items (1 selected)


admin@192.168.200.1 (MikroTik) - WinBox (64bit) v6.49.2 on hAP lite (smips)
Session Settings Dashboard
Safe Mode
Session:
Quick Set
CAPsMAN
Interfaces
Wireless
Bridge
PPP
Switch
Mesh
IP
MPLS
Routing
System
Queues
Files
Log
RADIUS
Tools
New Terminal
Make Supout.rif
New WinBox
Exit
Windows
RouterOS WinBox
Firewall
Filter Rules
NAT
Mangle
Raw
Service Ports
Connections
Address Lists
Layer7 Protocols
Reset Counters
Reset All Counters
Find
all
Dst. Port In. Inter... Out. In
;;; PC co
0
;;; PC Pa
1
;;; Other
2
;;; Other
3
Mangle Rule <>
General
Advanced
Extra
Action
Statistics
Action: mark packet
Log
Log Prefix:
New Packet Mark: PC Packet
Passthrough
OK
Cancel
Apply
Disable
Comment
Copy
Remove
Reset Counters
Reset All Counters
enabled
4 items (1 selected)

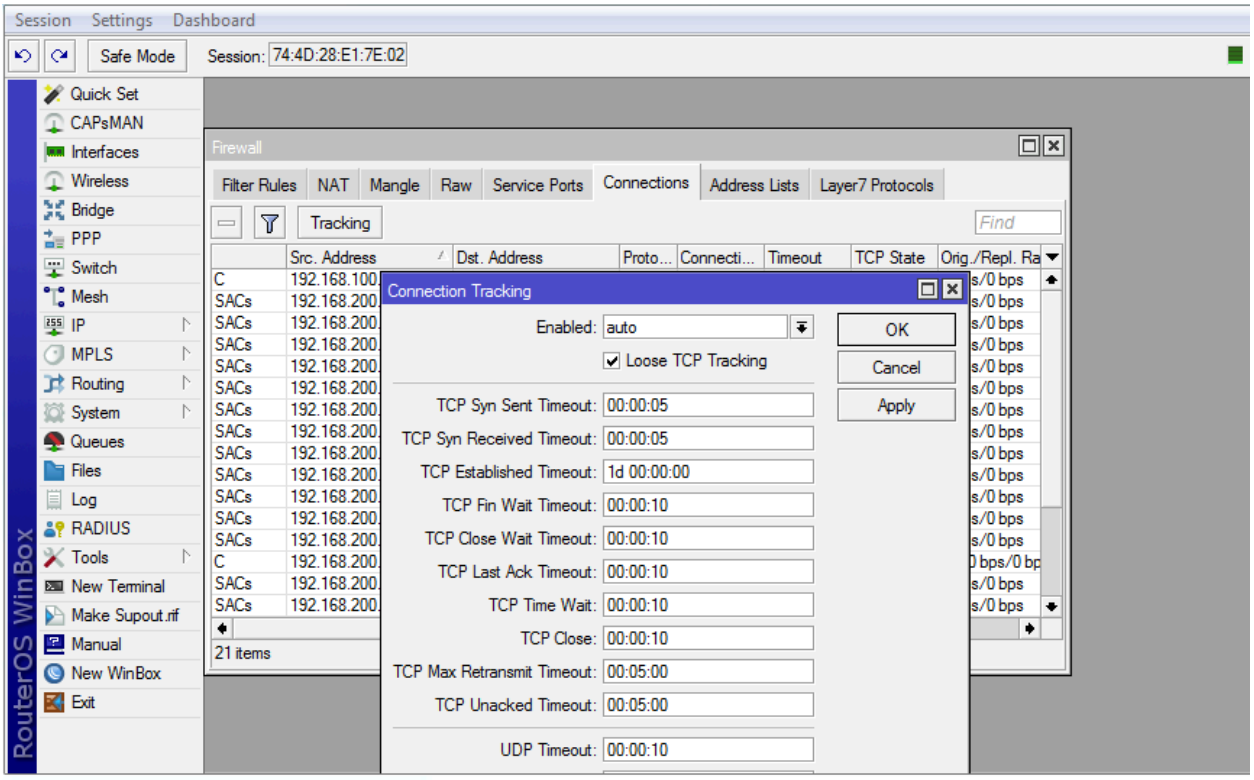

admin@192.168.200.1 (MikroTik) - WinBox (64bit) v6.49.2 on hAP lite (smips)
Session Settings Dashboard
Safe Mode
Session:
Quick Set
CAPsMAN
Interfaces
Wireless
Bridge
PPP
Switch
Mesh
IP
MPLS
Routing
System
Queues
Files
Log
RADIUS
Tools
New Terminal
Make Supout.rif
New WinBox
Exit
Windows
RouterOS WinBox
Firewall
Filter Rules
NAT
Mangle
Raw
Service Ports
Connections
Address Lists
Layer7 Protocols
Reset Counters
Reset All Counters
Find
all
Mangle Rule <>
General
Advanced
Extra
Action
Statistics
Chain: prerouting
Src. Address:
Dst. Address:
Protocol:
Src. Port:
Dst. Port:
Any. Port:
In. Interface:
Out. Interface:
In. Interface List:
Out. Interface List:
Packet Mark:
Connection Mark: PC connection
Routing Mark:
Routing Table:
Connection Type:
Connection State:
OK
Cancel
Apply
Disable
Comment
Copy
Remove
Reset Counters
Reset All Counters
enabled
4 items (1 selected)

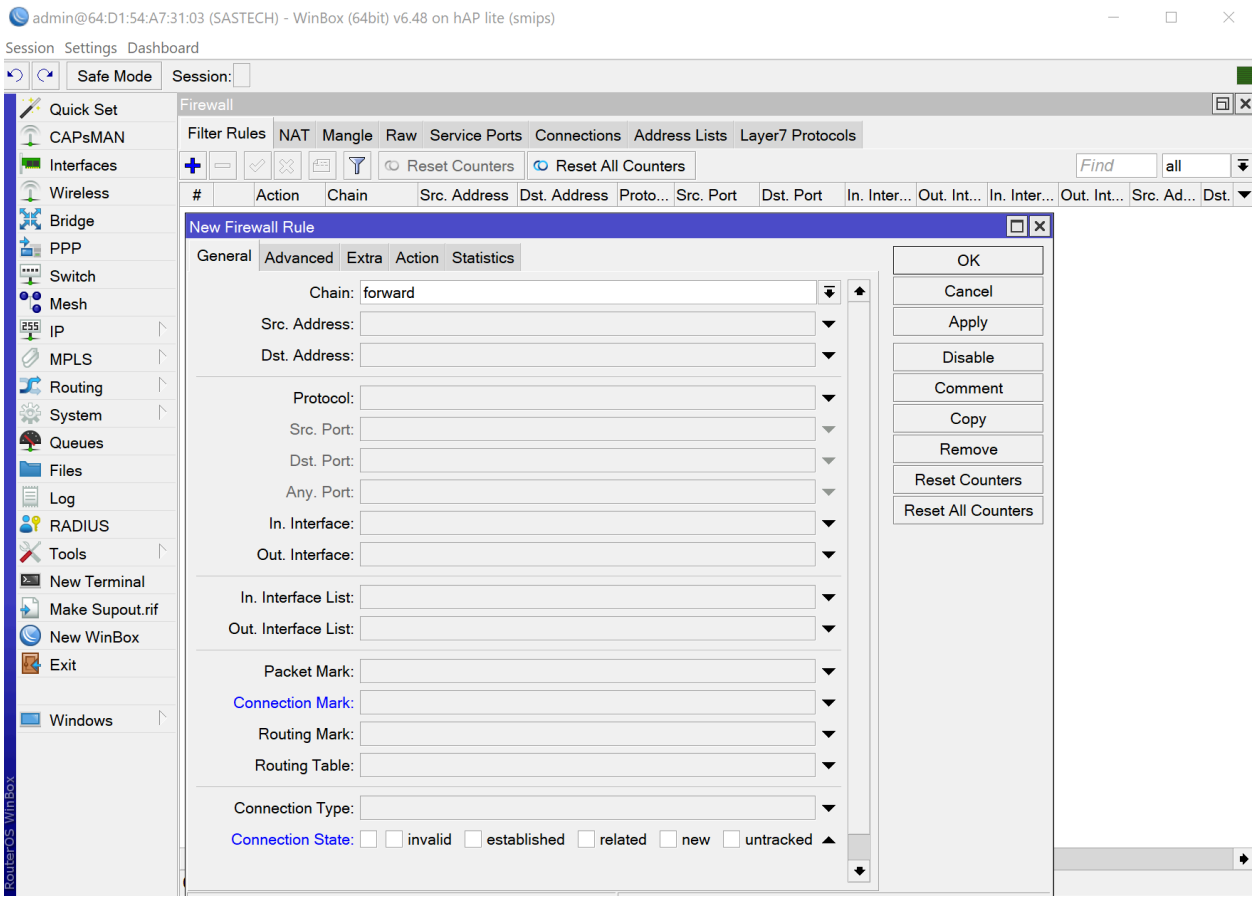


TTL change in mangle



Connection tracking


Session Settings Dashboard
Safe Mode
Session: 74:4D:28:E1:7E:02
RouterOS WinBox
Quick Set
CAPsMAN
Interfaces
Wireless
Bridge
PPP
Switch
Mesh
IP
MPLS
Routing
System
Queues
Files
Log
RADIUS
Tools
New Terminal
Make Supout.rif
Manual
New WinBox
Exit
Firewall
Filter Rules
NAT
Mangle
Raw
Service Ports
Connections
Address Lists
Layer7 Protocols
Tracking
Find
Src. Address
Dst. Address
Proto...
Connecti...
Timeout
TCP State
Orig./Repl. Ra
21 items
Connection Tracking
Enabled: auto
Loose TCP Tracking
OK
Cancel
Apply
TCP Syn Sent Timeout: 00:00:05
TCP Syn Received Timeout: 00:00:05
TCP Established Timeout: 1d 00:00:00
TCP Fin Wait Timeout: 00:00:10
TCP Close Wait Timeout: 00:00:10
TCP Last Ack Timeout: 00:00:10
TCP Time Wait: 00:00:10
TCP Close: 00:00:10
TCP Max Retransmit Timeout: 00:05:00
TCP Unacked Timeout: 00:05:00
UDP Timeout: 00:00:10

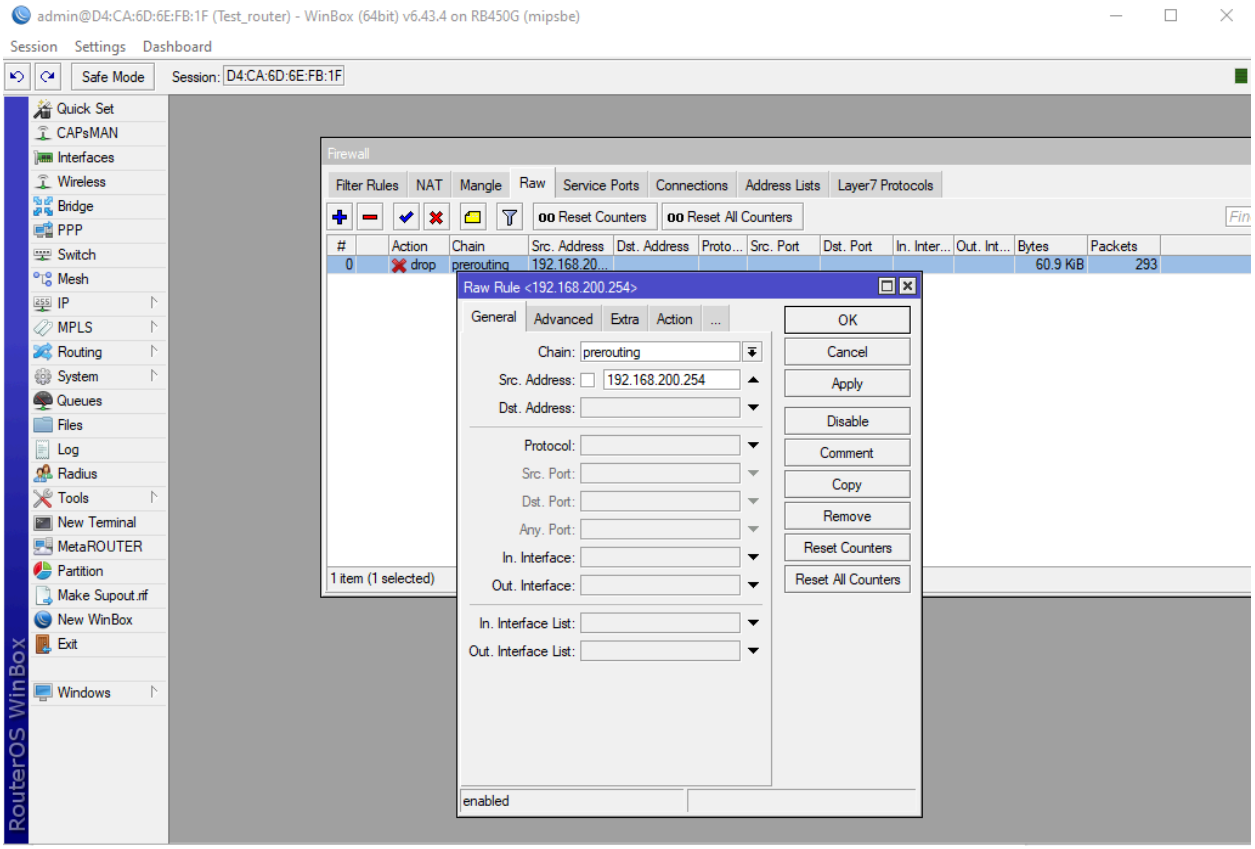


Connection tracking Example:

/ip firewall filter
add chain=input connection-state=invalid action=drop \
comment="Drop Invalid connections"

Raw
/ip firewall filter
add action=drop chain=forward src-address=192.168.200.254

/ip firewall raw
add action=drop chain=prerouting src-address=192.168.200.254

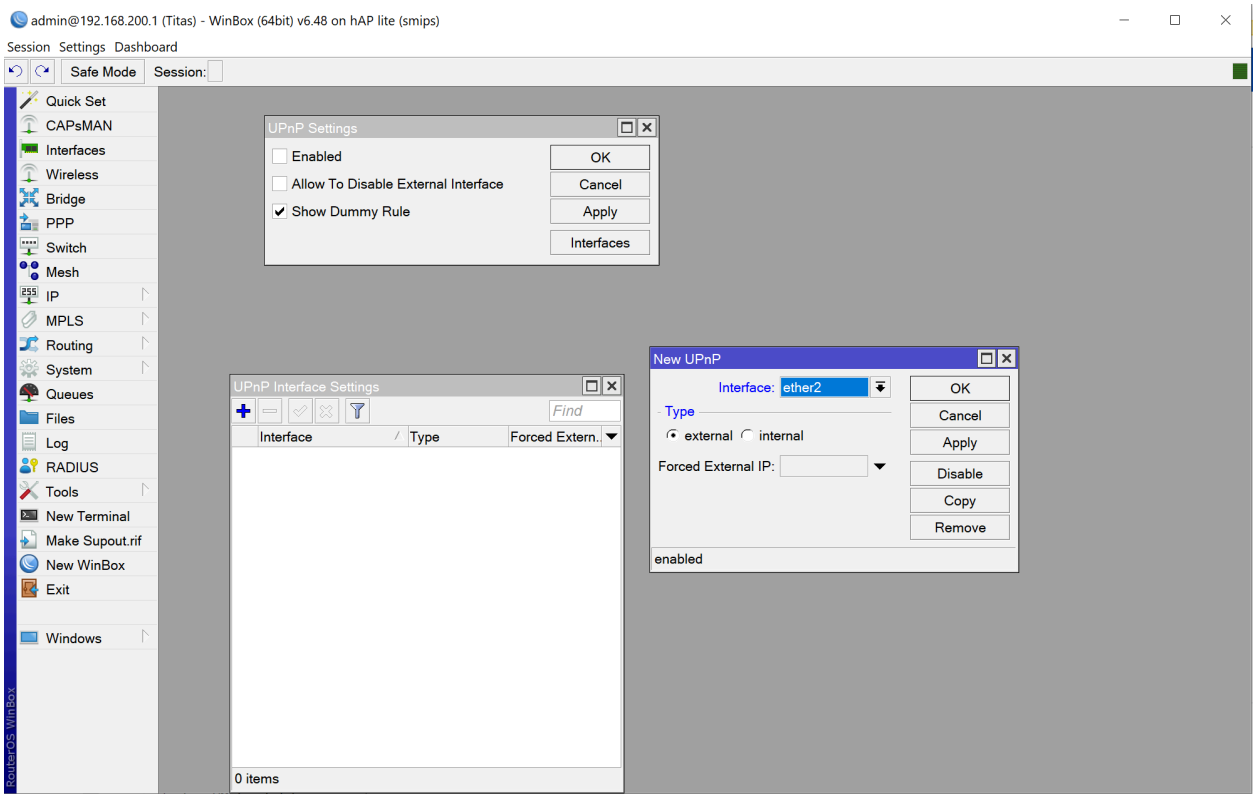


uPNP

Universal Plug and Play (UPnP) and NAT Port Mapping Protocol (NAT-PMP) are network services which allow software and devices to configure each other when attaching to a network. This includes automatically creating their own dynamic NAT port forwards and associated firewall rules

The UPnP protocol is used for many modern applications, like most of DirectX games, as well as for various Windows Messenger features (remote asisstance, application sharing, file transfer, voice, video) from behind a firewall.

/ip upnp

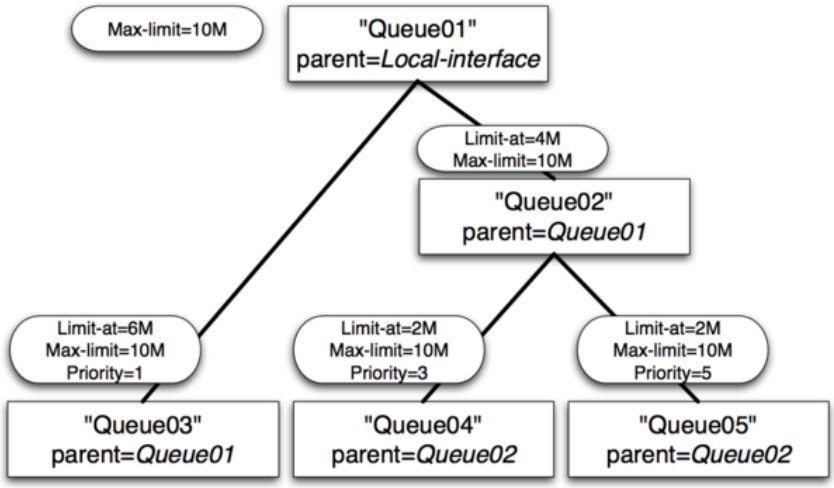


HTB general information

HTB (Hierarchical Token Bucket) is a classful queuing method that is useful for handling different kind of traffic. We have to follow three basic steps to create HTB:

Match and mark traffic – classify traffic for further use. Consists of one or more matching parameters to select packets for the specific class.
Create rules (policy) to mark traffic – put specific traffic class into specific queue and to define the actions that are taken for each class.
Attach policy for specific interface(-s) – append policy for all interfaces (global-in, global-out or global-total), for specific interface or for specific parent queue.

Example 1 : Usual case

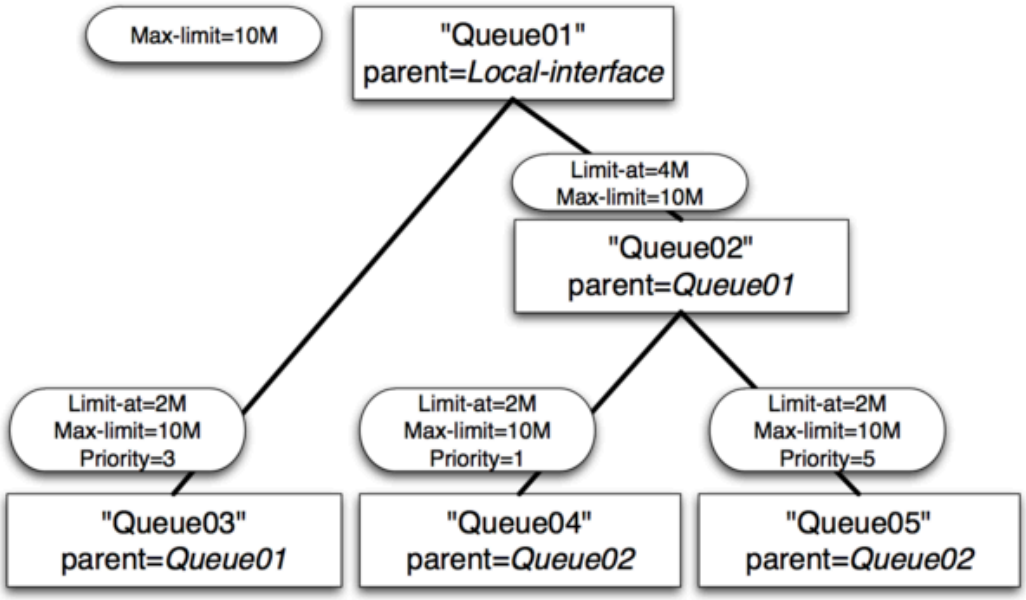


- **Queue01** limit-at=0Mbps max-limit=10Mbps
- **Queue02** limit-at=4Mbps max-limit=10Mbps
- **Queue03** limit-at=6Mbps max-limit=10Mbps priority=1
- **Queue04** limit-at=2Mbps max-limit=10Mbps priority=3
- **Queue05** limit-at=2Mbps max-limit=10Mbps priority=5

Result of Example 1

- **Queue03** will receive 6Mbps
- **Queue04** will receive 2Mbps
- **Queue05** will receive 2Mbps
- **Clarification:** HTB was build in a way, that, by satisfying all **limit-at**s, main queue no longer have throughput to distribute

Example 2 : Usual case with max-limit

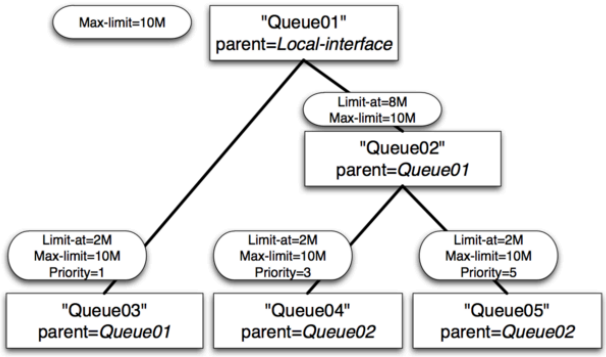


- **Queue01** limit-at=0Mbps max-limit=10Mbps
- **Queue02** limit-at=4Mbps max-limit=10Mbps
- **Queue03** limit-at=2Mbps max-limit=10Mbps priority=3
- **Queue04** limit-at=2Mbps max-limit=10Mbps priority=1
- **Queue05** limit-at=2Mbps max-limit=10Mbps priority=5

Result of Example 2

- **Queue03** will receive 2Mbps
- **Queue04** will receive 6Mbps
- **Queue05** will receive 2Mbps
- **Clarification:** After satisfying all **limit-at**s HTB will give throughput to queue with highest priority.

Example 3 : Inner queue limit-at

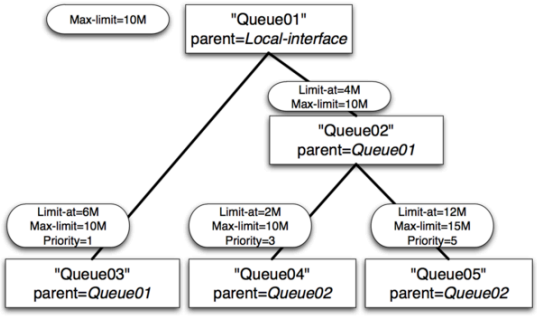


- **Queue01** limit-at=0Mbps max-limit=10Mbps
- **Queue02** limit-at=8Mbps max-limit=10Mbps
- **Queue03** limit-at=2Mbps max-limit=10Mbps priority=1
- **Queue04** limit-at=2Mbps max-limit=10Mbps priority=3
- **Queue05** limit-at=2Mbps max-limit=10Mbps priority=5

Result of Example 3

- **Queue03** will receive 2Mbps
- **Queue04** will receive 6Mbps
- **Queue05** will receive 2Mbps
- **Clarification:** After satisfying all **limit-at**s HTB will give throughput to queue with highest priority. But in this case **inner** queue **Queue02** had **limit-at** specified, by doing so, it reserved 8Mbps of throughput for queues **Queue04** and **Queue05**. From these two **Queue04** have highest priority, that is why it gets additional throughput.

Example 4 : Leaf queue limit-at



- **Queue01** limit-at=0Mbps max-limit=10Mbps
- **Queue02** limit-at=4Mbps max-limit=10Mbps
- **Queue03** limit-at=6Mbps max-limit=10Mbps priority=1
- **Queue04** limit-at=2Mbps max-limit=10Mbps priority=3
- **Queue05** limit-at=12Mbps max-limit=15Mbps priority=5

Result of Example 4

- **Queue03** will receive ~3Mbps
- **Queue04** will receive ~1Mbps
- **Queue05** will receive ~6Mbps
- **Clarification:** Only by satisfying all **limit-at**s HTB was forced to allocate 20Mbps - 6Mbps to **Queue03**, 2Mbps to **Queue04**, 12Mbps to **Queue05**, but our output interface is able to handle 10Mbps. As output interface queue is usually FIFO throughput allocation will keep ratio 6:2:12 or 3:1:6

HTB implementation (queue tree)

HTB structure
Check the presentation
https://mum.mikrotik.com/presentations/US09/Valens-MUM2009USA.pdf

HTB Dual Limitation
Dual Limitation
Each queue in HTB has two rate limits:

CIR (Committed Information Rate) – (limit-at in RouterOS) worst case scenario, flow will get this amount of traffic no matter what (assuming we can actually send so much data)
MIR (Maximal Information Rate) – (max-limit in RouterOS) best case scenario, rate that flow can get up to, if there queue's parent has spare bandwidth
In other words, at first limit-at (CIR) of the all queues will be satisfied, only then child queues will try to borrow the necessary data rate from their parents in order to reach their max-limit (MIR).

Note: CIR will be assigned to the corresponding queue no matter what. (even if max-limit of the parent is exceeded)

That is why, to ensure optimal (as designed) usage of dual limitation feature, we suggest to stick to these rules:

Sum of committed rates of all children must be less or equal to amount of traffic that is available to parent.
CIR(parent)* ≥ CIR(child1) +...+ CIR(childN)
*in case if parent is main parent CIR(parent)=MIR(parent)
Maximal rate of any child must be less or equal to maximal rate of the parent
MIR (parent) ≥ MIR(child1) & MIR (parent) ≥ MIR(child2) & ... & MIR (parent) ≥ MIR(childN)

HTB priority
Priority
We already know that limit-at (CIR) to all queues will be given out no matter what.

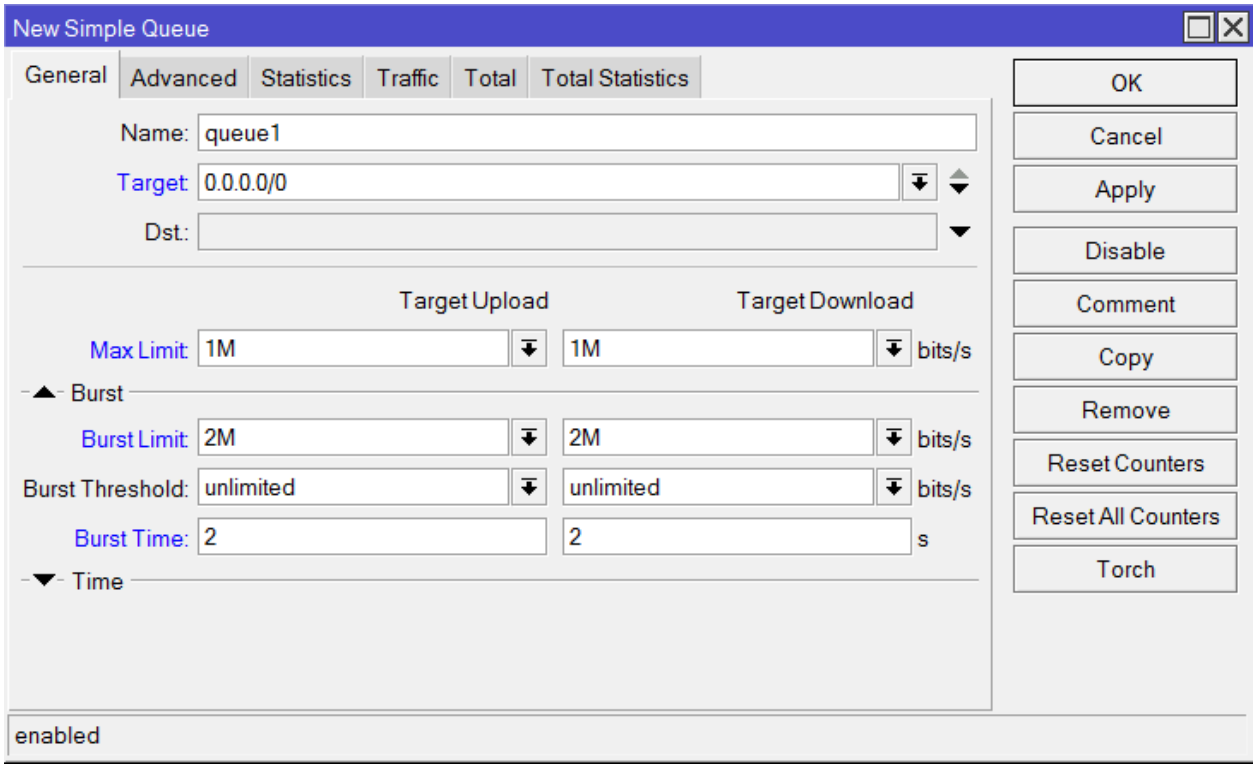
Priority is responsible for distribution of remaining parent queues traffic to child queues so that they are able to reach max-limit

Queue with higher priority will reach its max-limit before the queue with lower priority. 8 is the lowest priority, 1 is the highest.

Make a note that priority only works:

for leaf queues - priority in inner queue have no meaning.
if max-limit is specified (not 0)

Queues - Burst
Burst is a feature that allows to satisfy queue requirement for additional bandwidth even if required rate is bigger that MIR (max-limit) for a limited period of time.



Queue types

Kinds

Queue kinds are packet processing algorithms. Kind describe which packet will be transmitted next in the line. RouterOS supports the following Queueing kinds:

- FIFO (BFIFO, PFIFO, MQ PFIFO)
- RED
- SFQ
- PCQ

FIFO

These kinds are based on the FIFO algorithm (First-In-First-Out). The difference between **PFIFO** and **BFIFO** is that one is measured in packets and the other one in bytes. These queues use **pfifo-limit** and **bfifo-limit** parameters.

Every packet that cannot be enqueued (if the queue is full), is dropped. Large queue sizes can increase latency but utilize the channel better.

MQ-PFIFO is *pfifo* with support for multiple transmit queues. This queue is beneficial on SMP systems with ethernet interfaces that have support for multiple transmit queues and have a Linux driver support for multiple transmit queues (mostly on x86 platforms). This kind uses the **mq-pfifo-limit** parameter.

RED

Random Early Drop is a queuing mechanism that tries to avoid network congestion by controlling the average queue size. The average queue size is compared to two thresholds: a minimum (min_{th}) and maximum (max_{th}) threshold. If the average queue size (avg_q) is less than the minimum threshold, no packets are dropped. When the average queue size is greater than the maximum threshold, all incoming packets are dropped. But if the average queue size is between the minimum and maximum thresholds packets are randomly dropped with probability P_d where probability is exact a function of the average queue size: $P_d = P_{max}(avg_q - min_{th})/ (max_{th} - min_{th})$. If the average queue grows, the probability of dropping incoming packets grows too. P_{max} - ratio, which can adjust the packet discarding probability abruptness, (the simplest case P_{max} can be equal to one. The 8.2 diagram shows the packet drop probability in the RED algorithm.

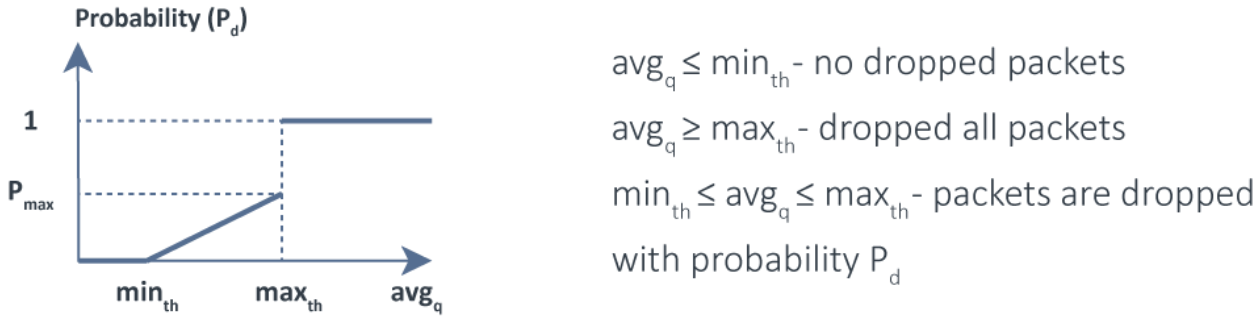


Figure 8.2. RED operation

SFQ

Stochastic Fairness Queuing (SFQ) is ensured by hashing and round-robin algorithms. SFQ is called "Stochastic" because it does not really allocate a queue for each flow, it has an algorithm that divides traffic over a limited number of queues (1024) using a hashing algorithm.

Traffic flow may be uniquely identified by 4 options (*src-address, dst-address, src-port,* and *dst-port*), so these parameters are used by the SFQ hashing algorithm to classify packets into one of 1024 possible sub-streams. Then round-robin algorithm will start to distribute available bandwidth to all sub-streams, on each round giving **sfq-allot** bytes of traffic. The whole SFQ queue can contain 128 packets and there are 1024 sub-streams available. The 8.3 diagram shows the SFQ operation:

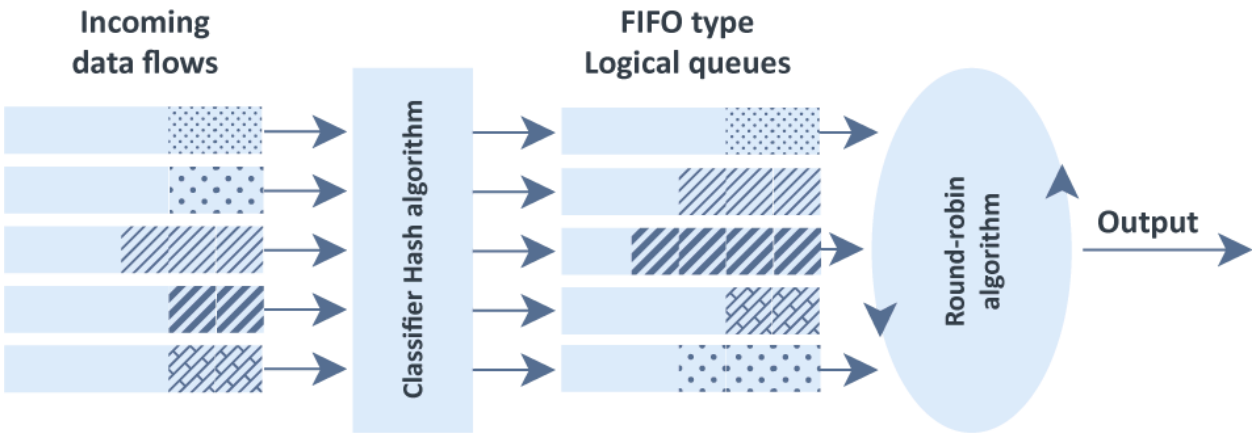


Figure 8.3. SFQ operation

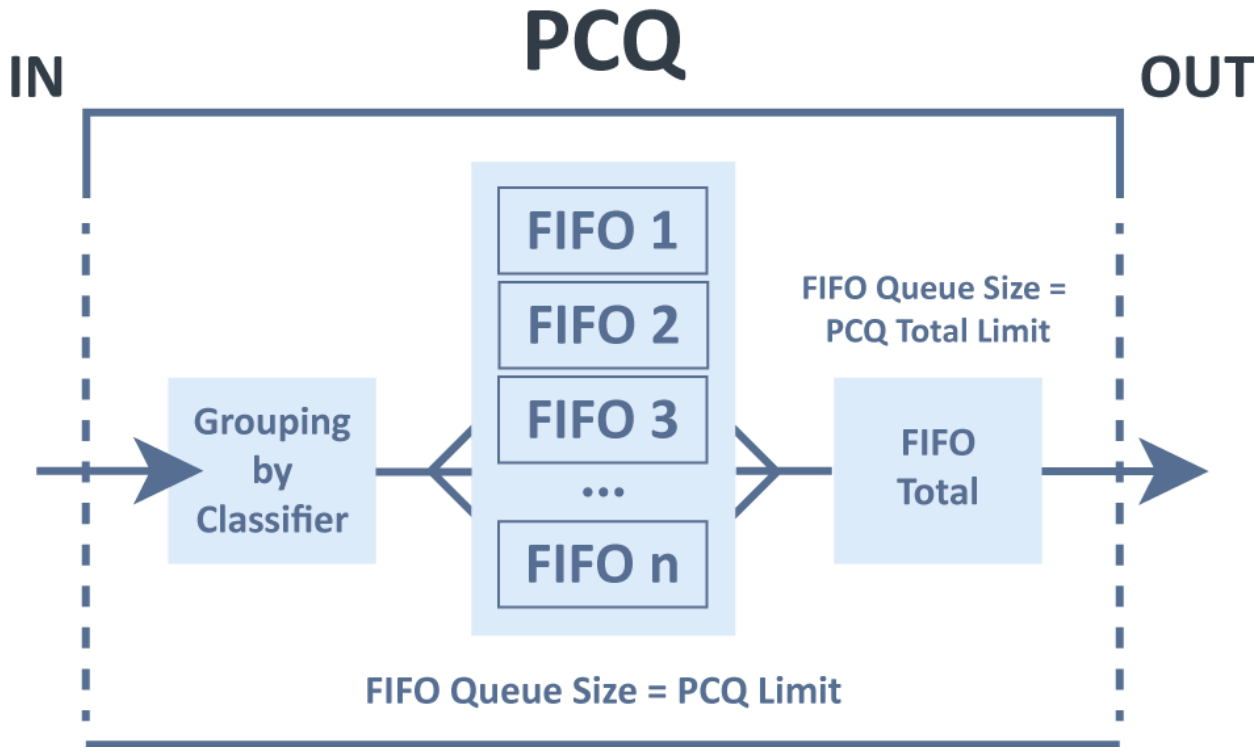
PCQ

PCQ algorithm is very simple - at first, it uses selected classifiers to distinguish one sub-stream from another, then applies individual FIFO queue size and limitation on every sub-stream, then groups all sub-streams together and applies global queue size and limitation.

PCQ parameters:

- **pcq-classifier** (dst-address | dst-port | src-address | src-port; default: "") : selection of sub-stream identifiers
- **pcq-rate** (number): maximal available data rate of each sub-steam
- **pcq-limit** (number): queue size of single sub-stream (in KiB)
- **pcq-total-limit** (number): maximum amount of queued data in all sub-streams (in KiB)

It is possible to assign a speed limitation to sub-streams with the **pcq-rate** option. If "pcq-rate=0" sub-streams will divide available traffic equally.



For example, instead of having 100 queues with 1000kbps limitation for download, we can have one PCQ queue with 100 sub-streams

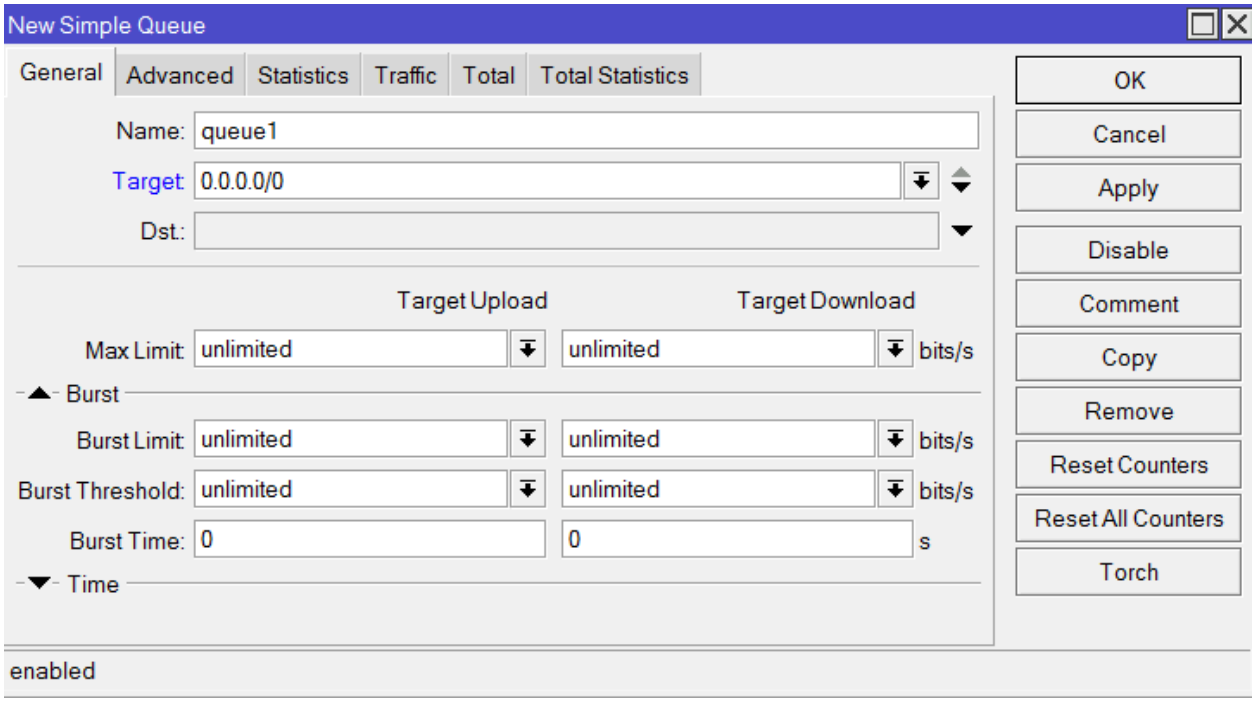
PCQ has burst implementation identical to Simple Queues and Queue Tree:

- **pcq-burst-rate** (number): maximal upload/download data rate which can be reached while the burst for substream is allowed
- **pcq-burst-threshold** (number): this is the value of burst on/off switch
- **pcq-burst-time** (time): a period of time (in seconds) over which the average data rate is calculated. (This is NOT the time of actual burst)

PCQ also allows using different size IPv4 and IPv6 networks as sub-stream identifiers. Before it was locked to a single IP address. This is done mainly for IPv6 as customers from an ISP point of view will be represented by /64 network, but devices in customers network will be /128. PCQ can be used for both of these scenarios and more. PCQ parameters:

- **pcq-dst-address-mask** (number): the size of the IPv4 network that will be used as a dst-address sub-stream identifier
- **pcq-src-address-mask** (number): the size of the IPv4 network that will be used as an src-address sub-stream identifier
- **pcq-dst-address6-mask** (number): the size of the IPV6 network that will be used as a dst-address sub-stream identifier
- **pcq-src-address6-mask** (number): the size of the IPV6 network that will be used as an src-address sub-stream identifier

Simple queues



Simple queue and queue tree interaction

Queue tree is handled before simple queues.

PCQ Queue mangle

Mangle rules

```
/ip firewall mangle
add action=mark-connection chain=prerouting comment=Up-conn \
    new-connection-mark=Up-conn passthrough=yes src-address=192.168.200.0/24
add action=mark-packet chain=prerouting comment=UP-PK connection-mark=Up-conn \
    new-packet-mark=UP-PK passthrough=no
add action=mark-connection chain=postrouting comment=Down-con dst-address=\
    192.168.200.0/24 new-connection-mark=Down-con passthrough=yes
add action=mark-packet chain=postrouting comment=down-pk connection-mark=\
    Down-con new-packet-mark=down-pk passthrough=no
```

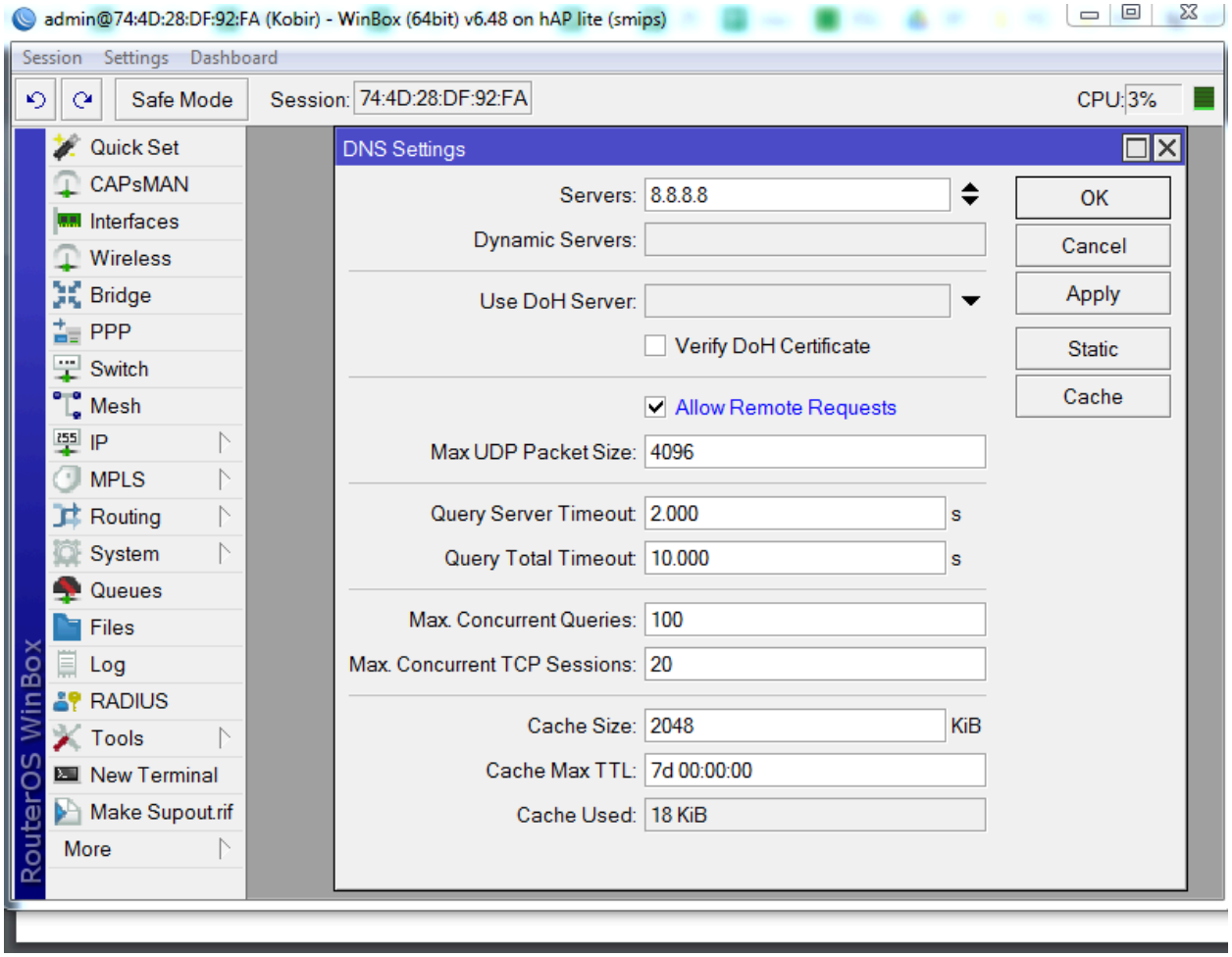
In the meantime create PCQ up and download

Queue tree rules

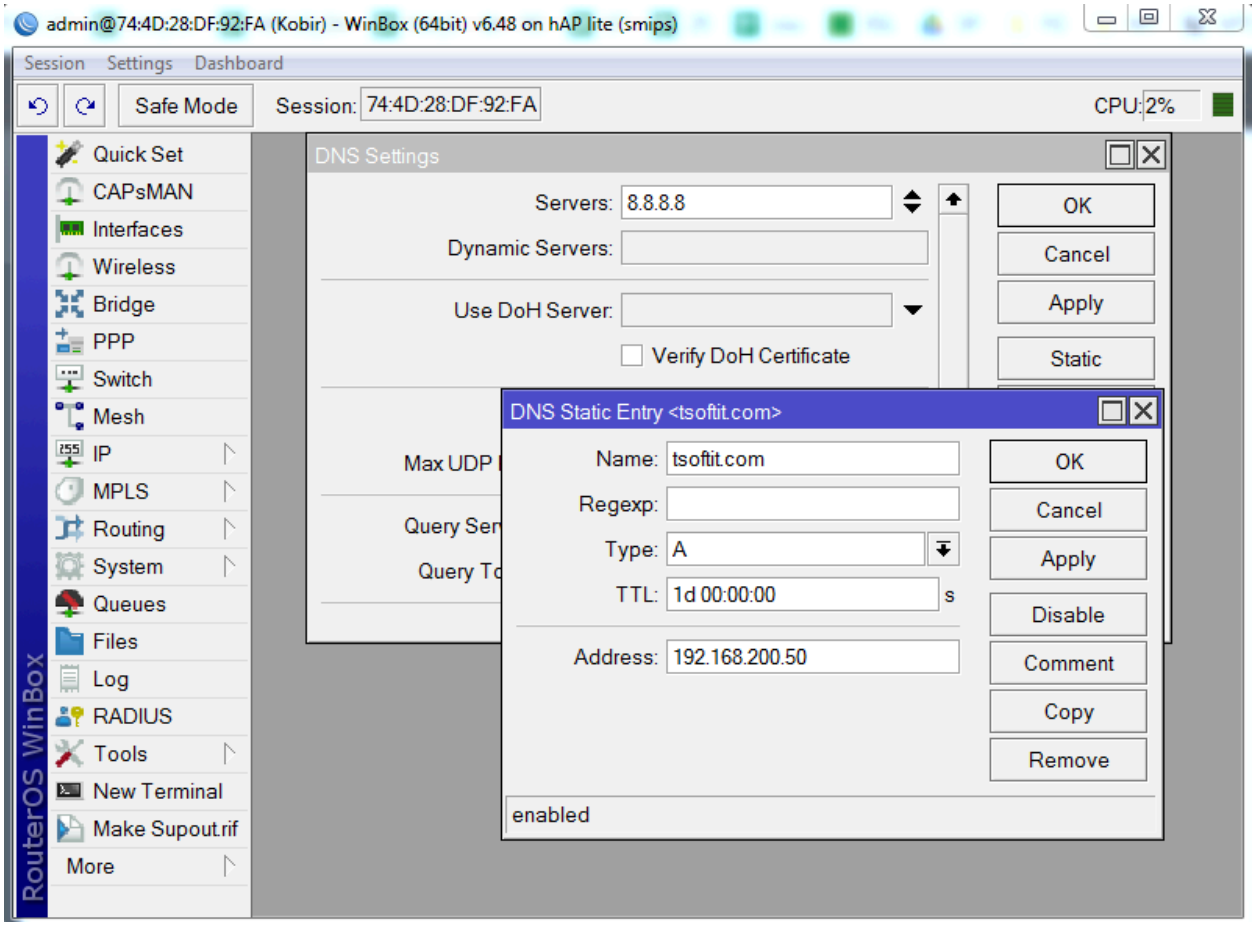
```
/queue tree
add max-limit=2M name="All Bangwidth" parent=global
add name=Down packet-mark=down-pk parent="All Bangwidth" queue=PCQ-Download
add name=UP packet-mark=UP-PK parent="All Bangwidth" queue=PCQ-up
```

DNS server issue and solutions

DNS client/cache

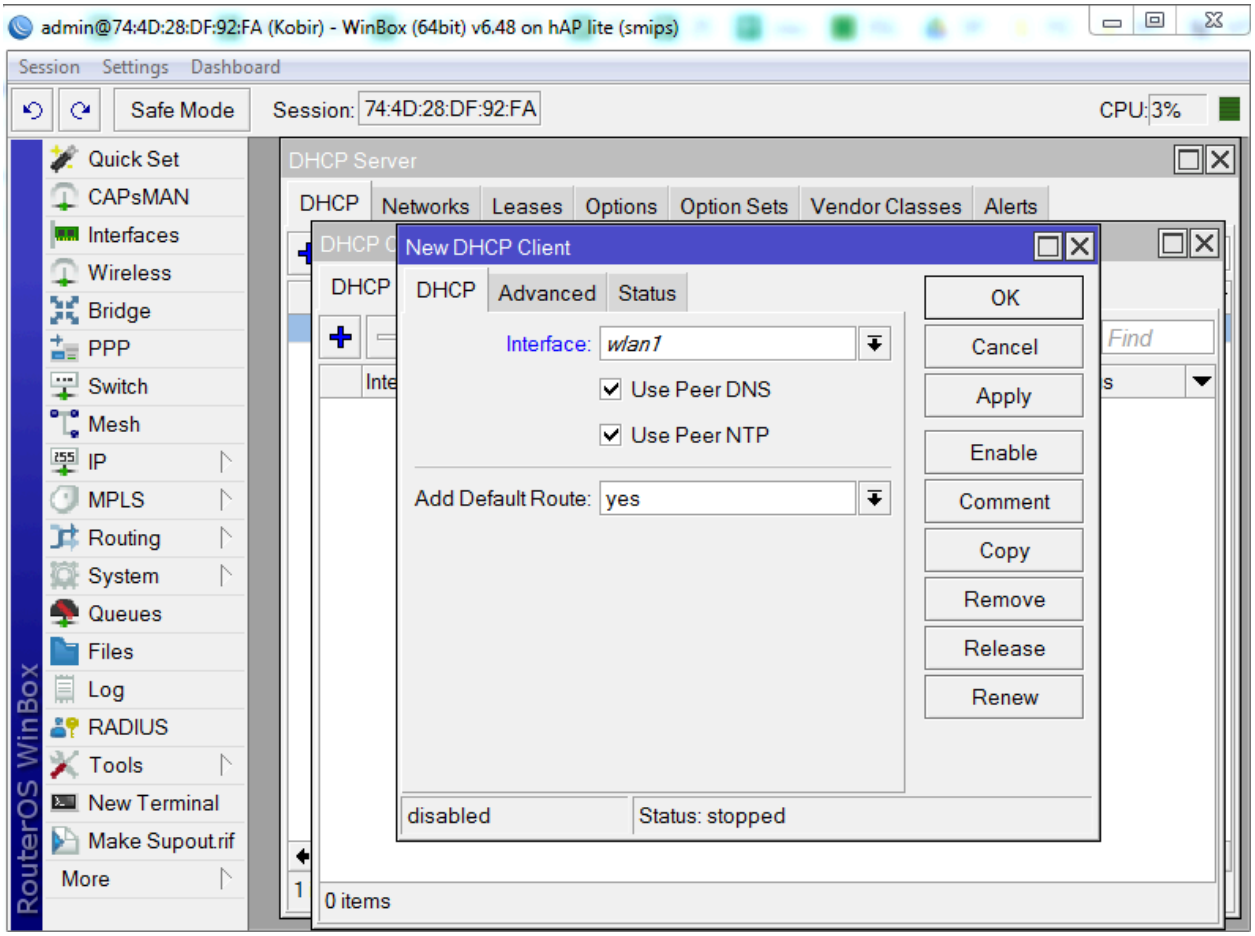


Static DNS
/ip dns static
add name=www.example.com address=10.0.0.1

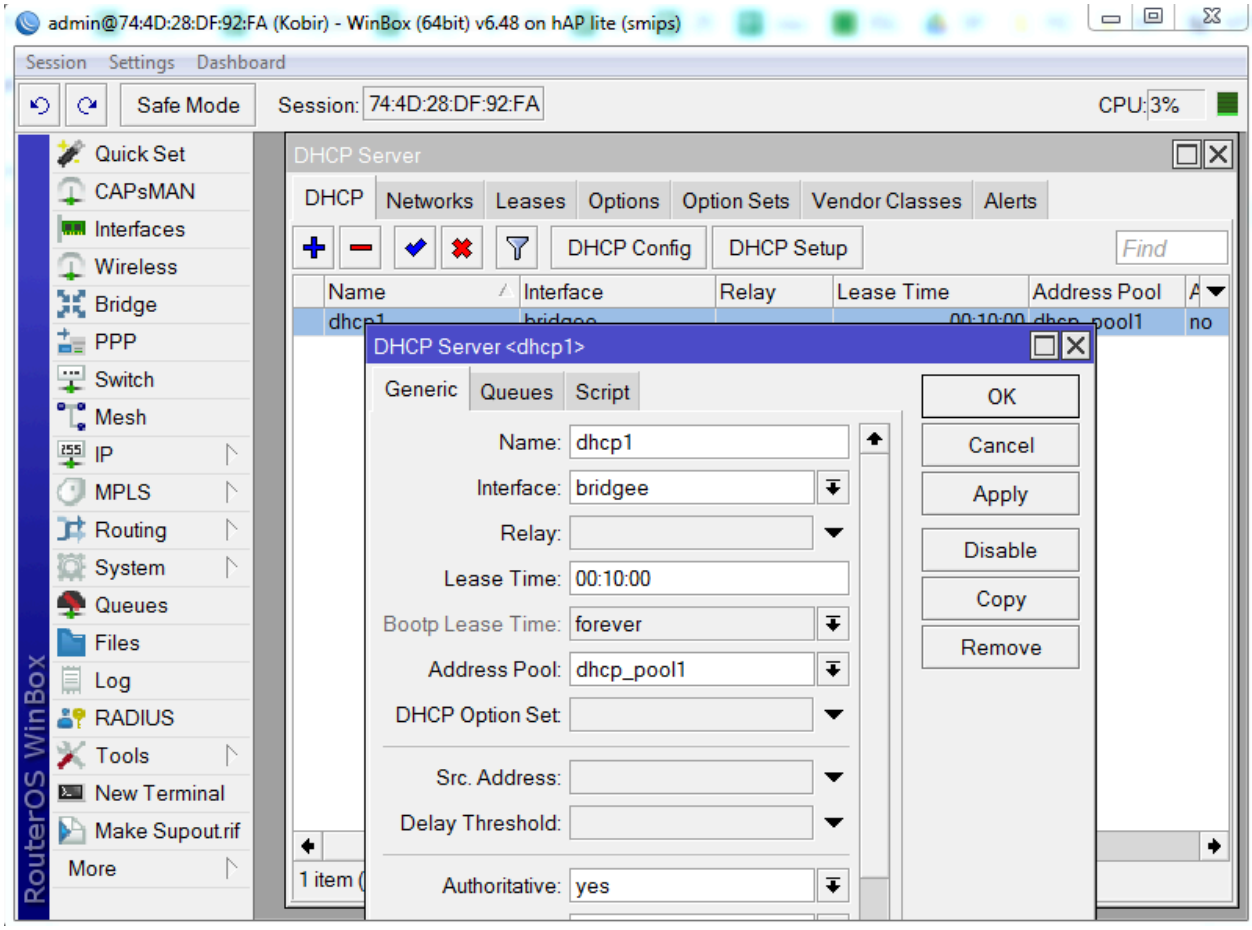


DHCP client/relay/server

DHCP client

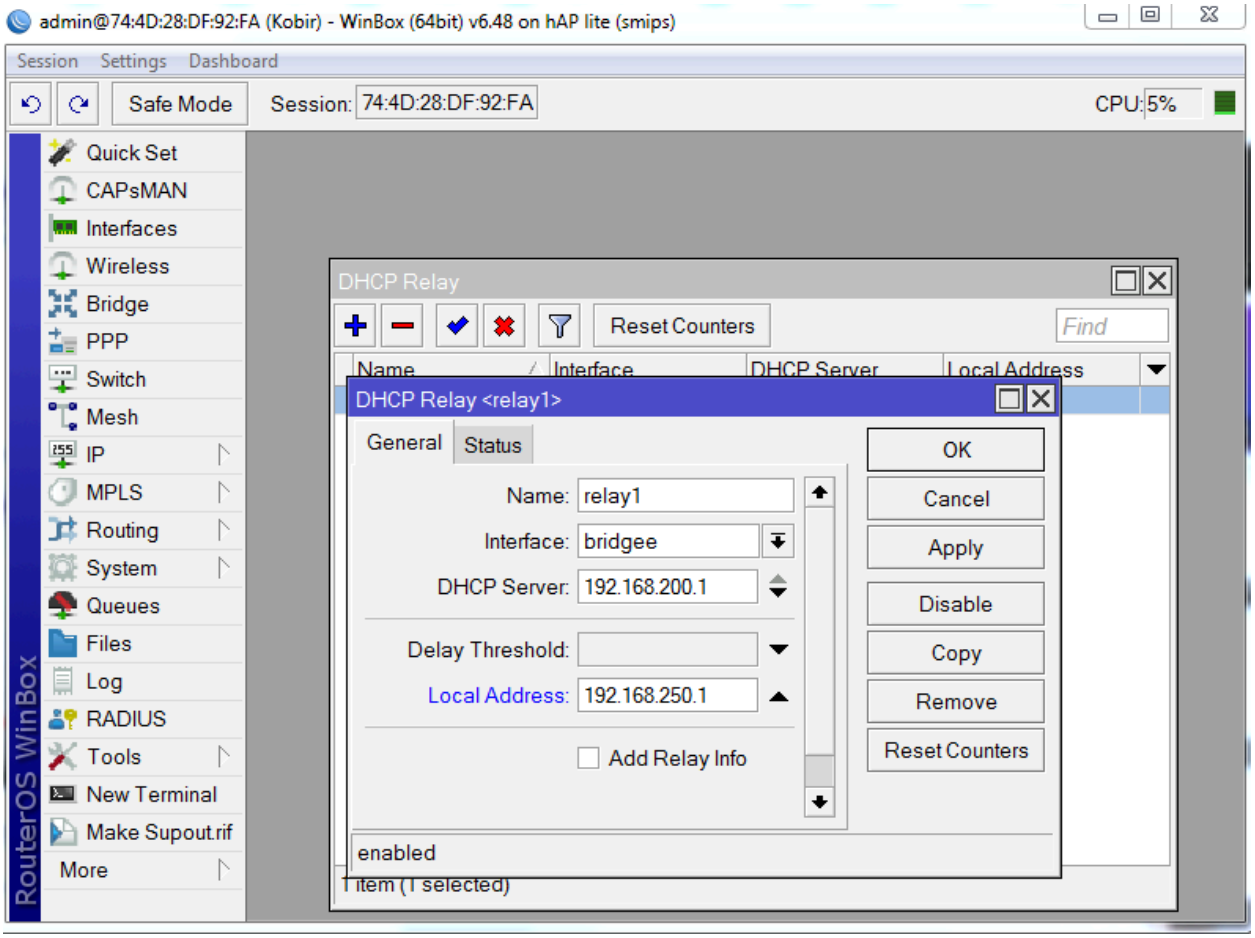


DHCP server



DHCP relay

DHCP Relay is just a proxy that is able to receive a DHCP request and resend it to the real DHCP server.



DHCP Alerts

To find any rogue DHCP servers as soon as they appear in your network, DHCP Alert tool can be used. It will monitor the ethernet interface for all DHCP replies and check if this reply comes from a valid DHCP server. If a reply from an unknown DHCP server is detected, alert gets triggered:
/ip dhcp-server alert

DHCP Options

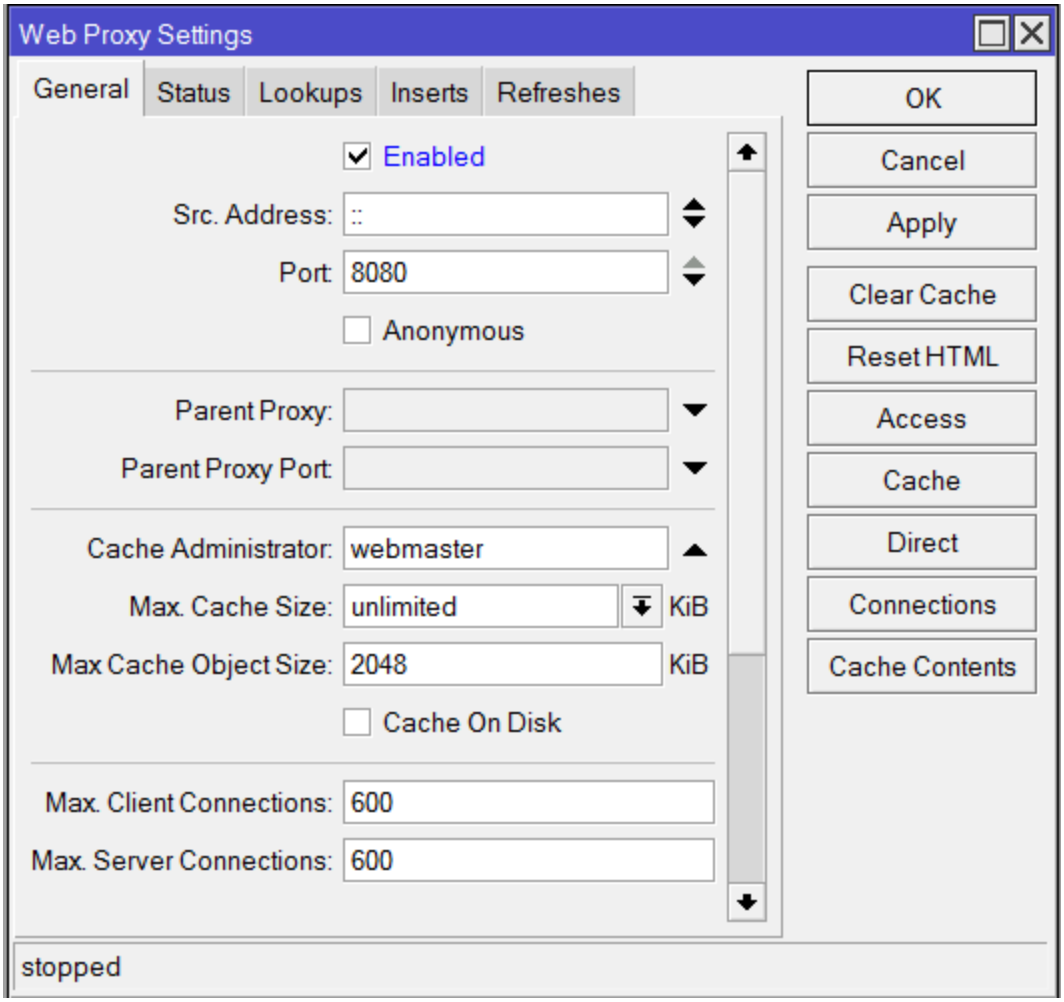
With the help of DHCP Option list, it is possible to define additional custom options for DHCP Server to advertise. Option precedence is as follows

example:
:/ip dhcp-server option
add code=66 name="option1" value="10.10.1.1"

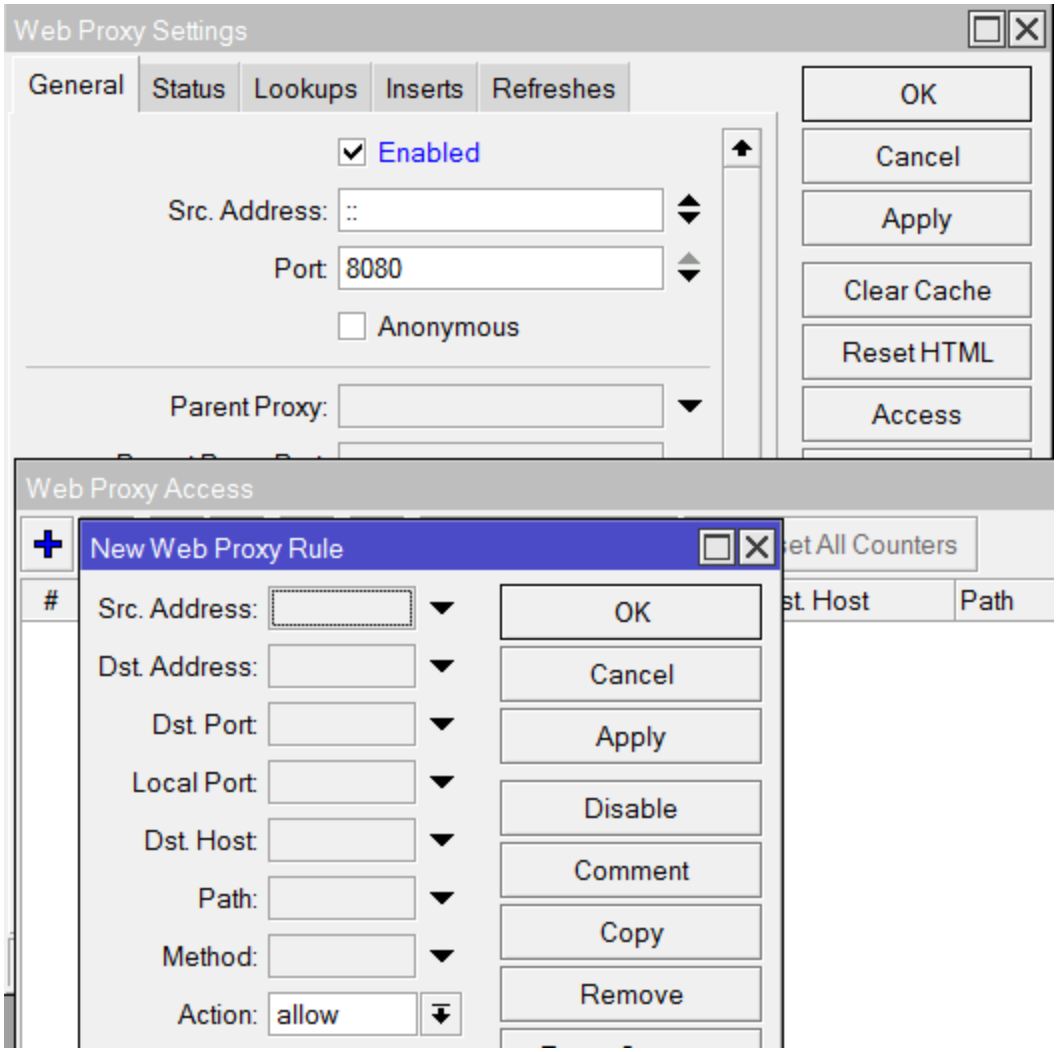
after that set option to use on dhcp network:
/ip dhcp-server network set <id> dhcp-option=option1

Now dhcp client will receive TFTP Server Name. Thats it

/ip dhcp-server option
Web Proxy
/ip firewall nat
add action=redirect chain=dstnat dst-port=80 protocol=tcp to-ports=8080



Access list



Regular expression
Regular expressions are also accepted here, but if the property should be treated as a regular expression, it should start with a colon (':').

To show that no symbols are allowed before the given pattern, we use ^ symbol at the beginning of the pattern.

To specify that no symbols are allowed after the given pattern, we use $ symbol at the end of the pattern.

Small hints in using regular expressions:

- \\ symbol sequence is used to enter \ character in console
- \. pattern means . only (in regular expressions single dot in pattern means any symbol)

- to show that no symbols are allowed before the given pattern, we use ^ symbol at the beginning of the pattern
- to specify that no symbols are allowed after the given pattern, we use $ symbol at the end of the pattern
- to enter [or] symbols, you should escape them with backslash \.

:log error "NOC Switch Down";